SKALA

Claude Code로 시작하는 바이브코딩 입문

- 말로 지시하고, 결과를 검증하고, 내 업무 도구로 남긴다

2일 과정 · Claude Code

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX



목차

과정 안내 — 과정 목표 · 실습 트랙 · 준비물

DAY 1 — 말로 지시해 첫 업무 도구를 만든다

1-1 바이브코딩 이해 · 1-2 환경 구성 · 1-3 첫 대화 · 1-4 지시문 · 1-5 검증과 오류 · 1-6 업무 도구 1호

DAY 2 — 도구를 에이전트로 넓힌다

2-1 에이전트 설계 · 2-2 도구 연동 · 2-3 Skill · Memory · 2-4 자동 실행 · 2-5 승인과 권한 · 2-6 종합 실습 · 2-7 현업 적용

부록 — 참고 자료와 추가 과제

A 명령 · 설치 요약 · B 템플릿 · C 용어 · D 추가 과제 · E 심화 · 참고 자료

과정 목표

내 업무 하나를
폴더 하나로 남기는 이틀

이 과정은 코드 작성법이 아니라 지시하고 검증하는 방법을 다룬다

Day 1 전반

이해와 준비

바이브코딩의 뜻과 한계

Claude Code 설치 ·
첫 대화

Day 1 후반

지시와 검증

지시문 · CLAUDE.md

Pass 기준 · 오류 해결

Day 2 전반

에이전트로 확장

도구 · Skill · Memory

계획을 먼저 받는 습관

Day 2 후반

자동화와 통제

예약 실행 · 승인 규칙

종합 실습과 회고

산출물은 지시문 · 규칙 · 도구가 든 폴더 하나이다

산출물이 쌓이는 순서

트랙 폴더에 한 가지씩 쌓이는 파일

앞 구간의 산출물이 다음 구간의 입력이 된다

Day 1 전반	Day 1 후반	Day 2 전반	Day 2 후반
설치 · 인증	CLAUDE.md	설계 문서	실행 스크립트
첫 결과물	Pass 기준	도구 스크립트	권한 규칙
	커맨드	Skill · 기억 파일	한 사이클 실행

구간이 끝날 때마다 해당 파일이 폴더에 있는지 확인한다

실습 트랙

실습은 직무 트랙 네 개 중 하나로 진행

절차는 같고 자료만 다르므로 본인 업무와 가장 가까운 트랙을 선택한다

트랙	폴더	Day 1 · 업무 도구	Day 2 · 에이전트
A · 영업	sales-quote	견적 문의 회신 초안	미회신 문의를 찾아 초안까지
B · 인사	hr-training	교육 이수 현황 리포트	미이수자 독려 초안까지
C · 재무 · 구매	fin-invoice	발주 · 세금계산서 대조표	불일치 건 확인 요청 초안까지
D · 품질 · IT 운영	ops-quality	불량 보고 주간 요약	반복 유형을 기억해 조치 요청 초안까지

트랙은 Day 1에 한 번 정하고 이틀 동안 변경하지 않는다

실습 폴더

트랙 폴더 하나에서 이틀 동안 작업

네 트랙의 폴더 구조는 동일하며, Day 2에 만드는 파일은 주석으로 표시했다

~/work/vibe/트랙 폴더

```
sales-quote/                # 트랙 A 예시
├── CLAUDE.md               # 규칙 (1-4)
├── data/                   # 원본 자료 · 수정 금지
├── inbox/                  # 새로 들어온 요청
├── report/                 # 결과물
├── outbox/drafts/          # 발송 전 초안
├── .claude/commands/       # 커맨드 (1-6)
├── .claude/skills/         # Skill (2-3)
├── tools/                  # 도구 스크립트 (2-2)
├── memory/                 # 기억 파일 (2-3)
└── scripts/                # 자동 실행 (2-4)
```

data/는 읽기만

원본 자료는 수정하지 않음

결과는 report/

매 실습의 산출물 위치

초안은 outbox/drafts/

사람이 확인한 뒤 발송

설정은 .claude/

커맨드 · Skill · 권한 규칙

폴더째 복사

과정 뒤 현업 업무에 재사용

시작 전 점검

설치 · 폴더 · 계정 · 트랙 확인

환경은 1-2에서 다시 점검하므로 지금은 문제가 있는 항목만 표시해 둔다

1

필수

Claude Code

claude --version 출력

claude 실행과 로그인

안 되면 1-2에서 설치

2

필수

실습 폴더

~/work/vibe에 압축 해제

트랙 폴더 네 개 확인

Python 3 설치 확인

3

필수

계정과 정책

Pro · Max · Team 등 유료 계정

사내 AI 사용 정책 확인

실제 고객 정보 입력 금지

4

권장

내 업무 후보

매주 반복하는 업무 하나

입력과 출력이 파일인 업무

2-7에서 사용

문제가 있으면 바로 알리고, 설치는 1-2에서 함께 진행한다

운영 원칙

이틀 동안 지킬 다섯 가지

Day 2에 설정 파일로 강제하기 전까지는 작업 습관으로 지킨다

원칙	내용	이유
원본은 읽기만	data/의 파일은 수정하지 않음	틀린 결과를 원본과 비교할 수 있어야 함
Pass 기준 먼저	지시하기 전에 확인할 문장 작성	기준이 없으면 완료 시점을 정할 수 없음
없는 값은 '확인 필요'	자료에 없는 수치를 만들지 않게 함	그럴듯한 값이 가장 찾기 어려움
발송은 사람이	초안까지만 맡기고 발송은 직접 함	되돌릴 수 없는 일은 사람이 결정
가상 데이터만	실제 고객 · 직원 정보는 넣지 않음	사내 정책 확인 전에는 넣을 수 없음

다섯 원칙은 Day 2의 2-5에서 권한 규칙으로 옮긴다

자주 나오는 질문

코딩 · 다른 도구 · 회사 자료 · 공유

Q 코딩을 전혀 몰라도 되나요?

가능하다. 코드는 Claude Code가 작성하고 사람은 결과를 확인해 수정할 점을 말한다.
터미널 명령 여덟 개와 파일 경로 읽는 법은 1-2에서 익힌다.

Q ChatGPT나 Gemini를 써도 되나요?

지시와 검증의 원리는 같다. 다만 실습 명령과 설정 파일은 Claude Code 기준이다.
다른 도구의 화면과 설정은 이 과정에서 다루지 않는다.

Q 회사 자료를 넣어도 되나요?

실습은 가상 데이터로 진행한다. 본인 업무에 적용할 때는 사내 AI 사용 정책을 먼저 확인한다.
이름 · 연락처 · 금액은 코드로 변경해 가명 처리한 뒤 연습한다.

Q 만든 것을 팀원도 쓸 수 있나요?

폴더를 복사하면 된다. 규칙 · 커맨드 · Skill · 권한 설정이 모두 파일이다.
공유 방법은 2-7에서 다룬다.

DAY 01

말로 지시해 첫 업무 도구 만들기

Day 1 · 1-1 ~ 1-6. Claude Code를 설치하고, 지시하고, 검증해 커맨드 하나로 남긴다

전반
1-1 ~ 1-3 · 이해 · 설치 · 첫 대화

후반
1-4 ~ 1-6 · 지시 · 검증 · 커맨드

마감
CLAUDE.md · 커맨드 파일 · 결과물 1건

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

오늘의 질문

무엇을 말로 지시하고,
무엇을 직접 확인할 것인가

Day 1이 끝날 때 본인 트랙 자료로 이 질문에 답할 수 있으면 된다

전반

지시할 준비

바이브코딩의 뜻과 한계
Claude Code 설치와 첫 실행
화면과 승인 창 읽기

후반 ①

지시하는 방법

역할 · 맥락 · 형식 · 제약
반복 지시는 CLAUDE.md로
한 번에 하나씩 수정하기

후반 ②

확인하는 방법

Pass 기준을 먼저 작성하기
숫자 · 날짜 · 이름부터 비교
오류는 원문 그대로 전달

Pass 기준: 새 대화에서 커맨드 한 줄로 같은 결과가 나온다

1-1

바이브코딩 이해

코드를 작성하지 않고 말로 만든다는 것의 뜻과 한계

01
정의
말로 지시하고 결과로 확인

02
범위
맞는 일과 안 맞는 일

03
원리
예측 · Hallucination · Context

개념 · 실습 없음

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

정의

바이브코딩은 코드 대신 의도 입력

만드는 사람의 일이 작성에서 지시와 확인으로 변경된다

기존

직접 작성

사람이 코드를 한 줄씩 작성

1. **입력**
   프로그래밍 언어
2. **필요한 것**
   문법과 도구 사용법
3. **막히는 곳**
   오류 메시지 해석
4. **결과 확인**
   작성자가 직접 테스트

→ 배우는 데 오래 걸림

→

바이브코딩

말로 지시

AI가 코드를 작성하고 실행

1. **입력**
   한국어 문장과 자료 파일
2. **필요한 것**
   업무 지식과 판단 기준
3. **막히는 곳**
   지시가 모호할 때
4. **결과 확인**
   Pass 기준으로 사람이 확인

→ 업무를 아는 사람이 유리하다

코드를 읽지 못해도 결과가 맞는지는 판단할 수 있어야 한다

사람의 역할

사람이 담당하는 일은 정의 · 판단 · 책임

AI가 초안과 코드를 만들어도 세 가지는 넘기지 않는다

역할	하는 일	이 과정에서
정의	무엇을 만들지, 어떤 자료로 만들지 정함	1-4 지시문 · CLAUDE.md
판단	결과가 맞는지, 사용할 수 있는지 확인	1-5 Pass 기준 · 검증
책임	발송 · 승인 · 확정처럼 되돌릴 수 없는 일 수행	2-5 승인과 권한

지시는 넘길 수 있어도 판단과 책임은 넘길 수 없다

적용 범위

잘 맞는 일과 맞지 않는 일

첫 과제는 왼쪽 조건을 만족하는 업무에서 선택한다

잘 맞는 일

입력과 출력이 파일이다

규칙을 말로 설명할 수 있는 반복 업무

- **자료 정리**
  여러 파일을 읽어 표 하나로 모음
- **대조**
  두 목록을 비교해 차이를 찾음
- **초안 작성**
  정해진 형식의 회신 · 보고 초안
- **작은 화면**
  혼자 사용하는 현황판 · 조회 화면

주의

틀리면 피해가 크다

사람의 확인 단계를 반드시 둠

- **외부 발송**
  메일 · 메시지를 자동으로 보내는 일
- **금액 확정**
  결제 · 정산 · 계약 금액
- **개인정보**
  실명 · 연락처 · 평가 자료
- **여럿이 사용하는 시스템**
  운영 중인 사내 시스템 수정

오른쪽 일도 금지 대상은 아니며, 초안까지만 맡기고 사람이 확정한다

작동 원리 ① 예측

AI는 정답을 찾지 않고 다음 말 예측

그럴듯한 문장을 만드는 능력과 사실을 아는 능력은 다르다

특성	뜻	지시에 주는 영향
다음 말 예측	앞 내용에 이어질 말을 확률로 선택	앞에 준 조건이 구체적일수록 결과가 좁혀짐
매번 조금 다름	같은 지시에도 문장이 달라짐	형식과 판정 값을 정해 주어야 비교할 수 있음
자료가 없으면 추정	모르는 부분도 자연스럽게 채움	'없으면 확인 필요'라고 작성해야 함
도구로 보완	파일을 읽고 명령을 실행해 확인	계산과 조회는 도구에 맡긴다 (2-2)

모호한 지시는 틀린 답이 아니라 '그럴듯한 답'으로 돌아온다

작동 원리 ② Hallucination

매끄러운 문장으로 나타나는 Hallucination

자료에 없는 수치 · 날짜 · 이름이 가장 흔한 형태이다

위험

막지 않은 지시

"지난달 실적을 요약해줘"

1. **자료에 없는 달**
   비슷한 값으로 채움
2. **합계**
   계산 과정 없이 숫자만 제시
3. **출처**
   어느 파일인지 표시 없음
4. **문장**
   매끄러워서 의심하기 어려움

→ 검토 없이 사용하면 사고 발생

대응

막는 지시

"data/에 없으면 '확인 필요'로 적어"

1. **자료에 없는 달**
   '확인 필요'로 표시됨
2. **합계**
   계산식을 함께 작성하게 함
3. **출처**
   파일명과 행을 작성하게 함
4. **확인**
   '확인 필요'만 검색하면 됨

→ 사람이 볼 곳이 좁혀짐

없는 값을 만들지 말라는 한 줄을 모든 지시에 넣는다

작동 원리 ③ Context Window

한 번에 볼 수 있는 양의 한도

대화가 길어지면 앞에서 준 조건이 반영되지 않는다

상황	일어나는 일	대응
대화가 길어짐	초반에 말한 형식 · 기준이 지켜지지 않음	/context로 사용량 확인 · /compact로 요약
주제가 변경됨	앞 주제의 내용이 다음 답에 섞임	/clear로 대화를 비우고 새로 시작
같은 조건 반복	매번 다시 입력하다가 빠뜨림	CLAUDE.md에 작성해 매 대화에 적용 (1-4)
큰 파일 통째로	한도의 대부분을 파일이 차지	필요한 파일만 @로 지정 (1-3)

길게 이어 가기보다 주제마다 대화를 새로 시작한다

Claude의 구성

Claude Code는 모델과 도구가 함께 일하는 구조

모델은 다음에 할 일을 정하고 도구는 실제 동작을 실행한다

구성	하는 일	확인하는 곳
모델	읽은 내용을 바탕으로 다음에 할 일을 정함	/model · /status
내장 도구	파일 작업 · 검색 · 명령 실행 · 웹 조회 실행	● Read · Write · Bash 줄
지침	CLAUDE.md와 설정이 모든 대화에 적용됨	/memory · /permissions
확장 기능	Skill · MCP · 서브에이전트로 능력을 더함	Day 2에서 다룸

같은 모델이라도 도구와 지침이 달라지면 결과가 달라진다

아는 것과 모르는 것

Claude가 아는 것과 직접 알려 줘야 하는 것

모르는 것은 묻지 않고 추정해서 답할 수 있으므로 정보의 출처를 구분한다

정보의 출처	예	반영할 점
학습한 지식	일반 상식 · 문서 작성법 · 프로그래밍 언어	기준일 이후의 변화는 반영되지 않음
이번 대화에서 읽은 것	@로 지정한 파일 · 명령의 출력	읽지 않은 파일의 내용은 모름
도구로 조회한 것	웹 검색 · 스크립트 실행 결과	최신 값과 계산은 조회하게 함
알려 주지 않으면 모르는 것	업무의 기준일 · 사내 규정 · 팀의 관행	CLAUDE.md에 작성한다 (1-4)

답의 근거가 학습한 지식인지 읽은 자료인지를 물어 구분한다

모델과 제품

Claude 모델과 제품의 구분

모델은 판단을 맡고 제품은 그 모델을 사용하는 화면이다

구분	이름	특징	이 과정에서
모델	Opus · Sonnet · Haiku 등	큰 모델일수록 어려운 판단에 강하고 사용량이 크다	/model (2-7)
제품	Claude 앱 (웹 · 데스크톱 · 모바일)	대화로 질문하고 답을 받음	다루지 않음
제품	Claude Code	내 폴더의 파일을 읽고 수정하고 명령 실행	이틀 동안 사용
제품	API · Agent SDK	프로그램에서 모델 호출	소개만

모델의 이름과 버전은 변경되므로 /model 화면에 표시되는 목록을 기준으로 한다

핵심 정리

1-1 정리 · 지시하고 확인하는 사람

바이브코딩
코드 대신 의도를 입력하고 결과로 확인

사람의 역할
정의 · 판단 · 책임

적용 범위
입력과 출력이 파일인 반복 업무부터

Hallucination
없는 값은 '확인 필요'로 표시하게 함

Context Window
주제마다 대화를 새로 시작

지시는 넘겨도
판단은 넘기지 않는다

NEXT
1-2 · Claude Code 환경 구성

가져갈 것

용어 세 가지는 이틀 동안 계속 나옴.
지시문 요령은 예측이라는 원리에서 나옴.
확인할 방법이 없는 일은 맡기지 않음.

1-2

Claude Code 환경 구성

터미널에서 Claude Code를 실행하기 위한 설치 · 인증 · 점검

01
터미널 기초
경로 · 필수 명령 8개

02
설치 · 인증
macOS · Windows 설치와 로그인

03
점검 · 문제 해결
claude doctor · 권한 모드

실습 L01

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

CLI가 필요한 이유

채팅 화면과 달리 파일과 명령을 직접 다루는 CLI

이 과정의 실습은 모두 터미널에서 진행한다

웹 채팅

대화 창 안에서 작업

결과를 사람이 복사해 옮김

1. **파일 입력**
   매번 올리거나 붙여 넣음
2. **결과 저장**
   사람이 복사해 파일로 저장
3. **규칙 유지**
   대화가 변경되면 다시 입력
4. **명령 실행**
   사람이 따로 실행

→ 한 번 묻고 답하는 일에 맞다

Claude Code

작업 폴더 안에서 작업

폴더의 파일을 직접 읽고 저장

1. **파일 입력**
   data/ 폴더를 직접 읽음
2. **결과 저장**
   report/ 폴더에 직접 저장
3. **규칙 유지**
   CLAUDE.md를 매 대화에 적용
4. **명령 실행**
   승인을 받아 직접 실행

→ 반복 업무를 맡기는 일에 맞다

파일을 읽고 저장하는 주체가 사람에서 Claude Code로 변경된다

사용 환경

여러 화면에서 같은 폴더를 다루는 Claude Code

과정은 터미널(CLI)에서 진행하며, CLAUDE.md와 설정 파일은 어느 화면에나 동일하게 적용된다

환경	특징	맞는 경우	이 과정에서
터미널 CLI	모든 기능을 명령으로 사용	반복 실행 · 자동화	실습 기준
VS Code · JetBrains	편집기 안에서 변경 내용을 나란히 확인	파일을 보면서 작업할 때	선택 사항
데스크톱 앱	설치형 화면에서 대화	터미널이 낯설 때	소개만
웹 · 모바일	브라우저와 앱에서 작업을 맡김	자리를 비웠을 때	소개만

화면이 달라도 지시하고 확인하는 방법은 같다

구성 요소

다섯 층으로 이루어진 실행 환경

아래층이 준비되어야 위층이 동작하므로 점검도 아래에서 위로 진행한다

층	macOS	Windows	확인
⑤ 작업 폴더	~/work/vibe/트랙 폴더	~/work/vibe/트랙 폴더	pwd · ls
④ 인증	Pro · Max · Team · Enterprise · Console	Pro · Max · Team · Enterprise · Console	claude auth status
③ Claude Code	네이티브 설치	네이티브 설치	claude --version
② 실행 도구	Python 3	Python 3 · Git for Windows	python3 --version
① 터미널	터미널 (zsh)	Git Bash	echo $SHELL

문제가 생기면 ①부터 차례로 확인한다

설치 전 확인

설치 전에 운영체제 버전과 계정 확인

요구 사항은 2026년 10월 공식 문서를 기준으로 정리했다

구분	macOS	Windows	확인 방법
운영체제	macOS 13.0 이상	Windows 10 1809 이상	macOS: sw_vers Windows: winver
하드웨어	RAM 4GB 이상 Apple Silicon · Intel	RAM 4GB 이상 x64 · ARM64	이 Mac에 관하여 설정 → 시스템 → 정보
사전 설치	없음	Git for Windows 권장	git --version
설치 권한	관리자 권한 불필요	관리자 권한 불필요	사용자 폴더에 설치
네트워크	인터넷 연결 필수	인터넷 연결 필수	사내망은 프록시 확인
계정	Pro · Max · Team · Enterprise Console	Pro · Max · Team · Enterprise Console	무료 플랜 불가

무료 claude.ai 플랜으로는 Claude Code에 로그인할 수 없다

운영체제별 터미널

운영체제에 따라 사용할 터미널부터 결정

Windows는 설치만 PowerShell에서 하고 실습은 Git Bash에서 한다

구분	macOS	Windows	공통
기본 터미널	터미널 (Terminal.app)	PowerShell (설치용) Git Bash (실습용)	VS Code 통합 터미널
여는 방법	Spotlight → '터미널'	시작 메뉴 → 'Git Bash'	VS Code → 보기 → 터미널
셸	zsh	bash	운영체제 기본 셸
홈 폴더	/Users/이름	C:\Users\이름 (Git Bash: /c/Users/이름)	~ 로 표기
프롬프트 모양	이름@Mac ~ %	PS C:\> 는 PowerShell $ 는 Git Bash	명령 실패 시 먼저 확인

Windows에서 Git Bash를 사용하면 명령 표기가 macOS와 같아진다

터미널 기초

직접 입력하는 터미널 명령은 여덟 개

명령을 외울 필요는 없으며, 막히면 Claude Code에 명령의 뜻을 질문한다

명령	기능	사용 예	사용 시점
pwd	현재 위치 확인	pwd	파일을 찾지 못할 때
ls	폴더 내용 보기	ls data	파일 이름 확인
cd	폴더 이동	cd ~/work/vibe/sales-quote	실습 시작 시
cd ..	상위 폴더로	cd ..	폴더를 잘못 들어갔을 때
mkdir -p	폴더 만들기	mkdir -p ~/work/vibe	작업 폴더 준비
open / start	파일 열기	open report/결과.html	결과 확인 (Windows: start)
Tab	경로 자동 완성	cd sales + Tab	긴 이름 입력
Ctrl + C	실행 중단	키 입력	명령이 중단하지 않을 때

경로는 직접 입력하지 말고 Tab으로 완성한다

운영체제별 명령

같은 동작도 운영체제와 셸에 따라 다른 명령

교재 표기는 macOS와 Git Bash 기준이다

동작	macOS · zsh	Windows · Git Bash	Windows · PowerShell
파일 열기	open report/결과.html	start report/결과.html	start report\결과.html
Python 실행	python3 tools/price.py	python tools/price.py 또는 p y	python tools\price.py 또는 py
명령 위치 확인	which claude	where claude	where.exe claude
홈 폴더	~ (/Users/이름)	~ (/c/Users/이름)	$HOME (C:\Users\이름)
붙여넣기	Cmd + V	Shift + Insert	Ctrl + V
환경 변수 지정	export 이름=값	export 이름=값	$env:이름 = "값"

명령이 실패하면 현재 창이 Git Bash인지 PowerShell인지부터 확인한다

경로 규칙

작업 폴더는 영문 · 공백 없는 짧은 경로에

교재의 모든 경로는 ~/work/vibe를 기준으로 작성했다

권장 위치와 피할 위치

```
~/work/vibe/                  # 과정 작업 폴더
├── sales-quote/              # 트랙 A
├── hr-training/              # 트랙 B
├── fin-invoice/              # 트랙 C
└── ops-quality/              # 트랙 D

# 피해야 할 위치
~/Desktop/새 폴더 (2)/실습    # 공백 · 괄호 · 한글
~/OneDrive - 회사명/문서/...   # 동기화 폴더
C:\Program Files\...          # 관리자 권한 필요
```

~는 홈 폴더

/Users/이름 · /c/Users/이름

절대 경로와 상대 경로

/ 또는 ~로 시작하면 절대 경로

공백 · 괄호 · 한글 금지

따옴표가 빠지면 경로가 둘로 나뉨

동기화 폴더 금지

저장 도중 파일이 잠길 수 있음

한 번에 하나

터미널 창 하나에 트랙 폴더 하나

설치 방법

Claude Code 설치의 기본은 네이티브 설치

방법에 따라 사전 요구 사항과 업데이트 방식이 다르다

방법	명령	사전 요구	특징
네이티브 macOS	curl -fsSL https://claude.ai/install.sh | bash	없음	권장 · 자동 업데이트
네이티브 Windows PowerShell	irm https://claude.ai/install.ps1 | iex	Git for Windows 권장	권장 · 자동 업데이트
네이티브 Windows CMD	curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd	위와 같음	PowerShell 사용 불가 시
Homebrew macOS	brew install --cask claude-code	Homebrew	수동 업데이트 (brew upgrade)
WinGet Windows	winget install Anthropic.ClaudeCode	WinGet	수동 업데이트 (winget upgrade)
npm 공통	npm install -g @anthropic-ai/claude-code	Node.js 22+	sudo 사용 금지

macOS와 Windows 모두 첫 번째 방법을 사용한다

설치 · macOS

설치부터 버전 확인까지 명령 세 줄

명령을 입력 순서대로 정리했으며, 출력 문구는 버전에 따라 다르다

Terminal · zsh

```
curl -fsSL https://claude.ai/install.sh  bash
내려받는 동안 진행 표시가 없다

새 터미널 창을 연 뒤
$ claude --version
버전 (Claude Code) 형식으로 출력

$ which claude
  /Users/이름/.local/bin/claude

버전이 출력되면 설치 완료. 다음은 로그인
$ claude
```

① 설치 스크립트
~/.local/bin에 설치

② 새 터미널
기존 창은 PATH가 반영되지 않음

③ 버전 확인
버전 번호가 나오면 정상

④ 설치 위치 확인
경로가 둘이면 설치 중복

⑤ 첫 실행
브라우저 로그인으로 이동

설치 절차 · Windows

Windows는 Git for Windows를 먼저 설치

Git Bash가 있으면 Claude Code가 bash로 명령을 실행하므로 교재 명령도 bash 기준이다

STEP 1

Git for Windows

git-scm.com에서 설치
기본 옵션 유지
Git Bash가 함께 설치됨

→ git --version

➡

STEP 2

Claude Code 설치

PowerShell 실행
irm https://claude.ai/install.ps1 | iex
관리자 권한 없이 실행

→ 설치 완료

➡

STEP 3

새 창에서 확인

Git Bash 새 창 실행
claude --version
where claude

→ 버전 출력

➡

STEP 4

경로 지정

Git Bash를 찾지 못할 때만
settings.json의 env 항목
CLAUDE_CODE_GIT_BASH_PATH

→ 필요 시에만

Windows의 Python 명령은 python 또는 py일 수 있다

설치 · Windows

Windows는 PowerShell에서 설치하고 Git Bash에서 확인

명령을 입력 순서대로 정리했으며, 출력 문구는 Windows와 Git 버전에 따라 다르다

PowerShell → Git Bash

```
PowerShell 창 (프롬프트가 PS 로 시작)
PS> git --version
git version … 이 출력되면 STEP 1 완료
PS> irm https://claude.ai/install.ps1 | iex
내려받는 동안 진행 표시가 없다

Git Bash 새 창을 연 뒤
$ claude --version
버전 (Claude Code) 형식으로 출력
$ where claude
  C:\Users\이름\.local\bin\claude.exe

$ claude
```

① Git 확인
없으면 git-scm.com에서 설치

② 설치 스크립트
관리자 권한 없이 실행

③ 새 Git Bash 창
기존 창은 PATH가 반영되지 않음

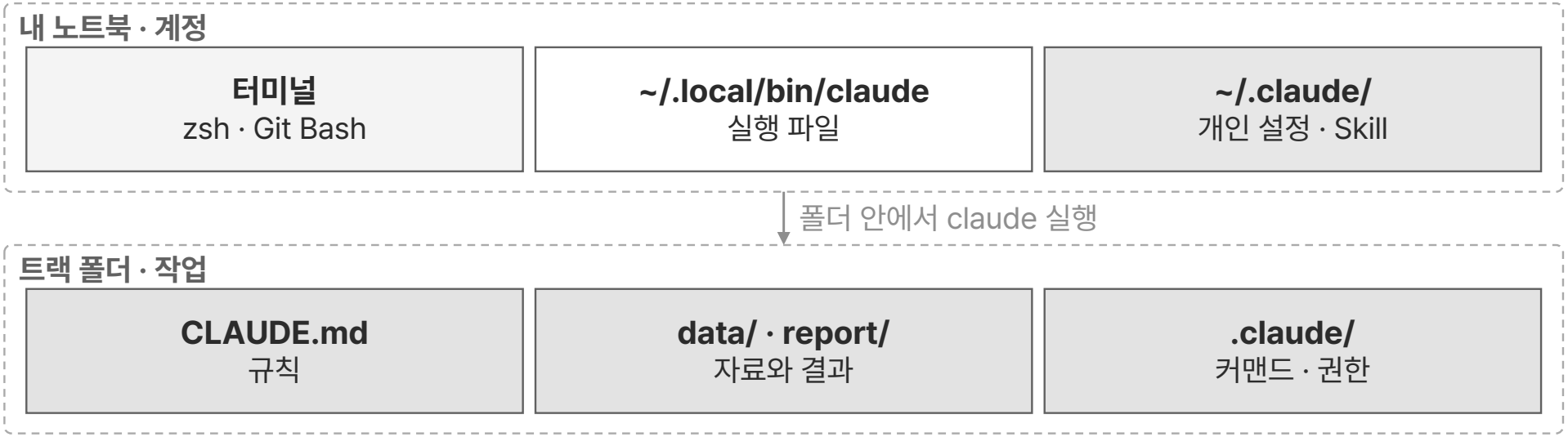
④ 설치 위치 확인
사용자 폴더의 .local\bin

⑤ 첫 실행
브라우저 로그인으로 이동

설치 후 구조

설치는 홈 폴더에, 작업은 트랙 폴더에

설치는 계정당 한 번이고, 트랙 폴더는 업무마다 따로 둔다



실습에서 수정하는 것은 트랙 폴더 안의 파일뿐이다

인증

첫 실행 때 브라우저에서 계정 인증

인증 정보는 노트북에 저장되고 다음 실행부터는 묻지 않는다

STEP 1

claude 실행

트랙 폴더에서 claude 입력
화면 테마 선택
로그인 방식 선택

→ 브라우저 열림

STEP 2

브라우저 로그인

Claude 계정으로 로그인
승인 버튼 선택
터미널로 돌아오기

→ 인증 완료

STEP 3

폴더 신뢰

처음 여는 폴더는 신뢰 확인
실습 폴더는 Yes 선택
트랙 폴더마다 한 번

→ 입력창 표시

STEP 4

상태 확인

claude auth status --text
대화 중에는 /status
계정 전환은 /login

→ 계정 표시

브라우저가 열리지 않으면 c 키로 URL을 복사해 붙여 넣는다

권한 모드

세션 시작은 auto 모드, 실습은 manual 모드

현재 모드는 입력창 아래 표시로 확인하며, Shift+Tab을 누를 때마다 아래 순서로 변경된다

화면 표시	모드	묻지 않고 실행하는 범위	이 과정에서
▸▸ auto mode on	auto	전부. 분류 모델이 동작마다 검토	시작 모드. Shift+Tab 1회로 해제
⏸ manual mode on	default	읽기만	실습 기본. 승인 창을 직접 읽음
▸▸ accept edits on	acceptEdits	읽기 · 파일 수정 · mkdir 등	반복 수정 단계에서만
⏸ plan mode on	plan	읽기. 수정 없이 계획만 제시	2-1에서 사용

claude --permission-mode default로 시작하면 처음부터 manual 모드이다

auto 모드

auto 모드에서도 실행 전에 차단되는 동작

분류 모델이 동작마다 검토하며, 제공 범위는 요금제와 버전에 따라 다르다

구분	동작	예
기본 차단	내려받은 스크립트를 바로 실행	curl … | bash
기본 차단	민감한 정보를 밖으로 전송	API 키 · 인증 정보
기본 차단	대량 삭제 · 운영 배포 · 권한 변경	rm -rf · 배포 명령
기본 허용	폴더 안의 파일 읽기 · 수정	report/ 저장
기본 허용	읽기 전용 조회	문서 페이지 읽기

차단이 이어지면 auto 모드는 사람에게 묻는 방식으로 돌아간다

문제 해결 · 공통

설치 오류는 증상 문구로 원인 좁히기

공통 · macOS 증상 여섯 가지를 정리했으며, Windows 증상은 다음 장에서 다룬다

증상	원인	조치
command not found: claude	PATH 미반영	새 터미널 실행 → 계속되면 echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
npm EACCES 권한 오류	전역 설치 권한 부족	sudo 사용 금지 · 네이티브 설치로 전환
claude가 두 경로에서 발견됨	설치 방법 중복	which -a claude로 확인 → 사용하지 않는 쪽 제거
설치 스크립트 연결 실패	사내 프록시	HTTPS_PROXY=http://프록시주소:포트 설정 후 재실행
인증서 오류 (certificate)	사내 보안 인증서	NODE_EXTRA_CA_CERTS=인증서파일.pem 설정
로그인 브라우저가 열리지 않음	팝업 차단 · 원격 접속	터미널에서 c 키로 URL 복사 → 브라우저에 붙여넣기

프록시와 인증서 값은 사내 IT 부서에 문의한다

문제 해결 · Windows

Windows 설치 오류는 어느 창에서 실행했는지부터 확인

프롬프트가 PS C:\이면 PowerShell, C:\이면 CMD, $이면 Git Bash이다

증상	원인	조치
'irm' is not recognized ...	CMD 창에서 PowerShell 명령 실행	PowerShell 창을 열어 같은 명령을 다시 실행
The token '&&' is not a valid statement separator	PowerShell 창에서 CMD 명령 실행	irm https://claude.ai/install.ps1 | iex 사용
claude 명령을 찾지 못함	PATH에 .local\bin 없음	새 창 실행 → 계속되면 사용자 PATH에 %USERPROFILE%\.local\bin 추가
Git Bash를 찾을 수 없음	Git 미설치 · 비표준 경로	where.exe git으로 위치 확인 → settings.json env에 CLAUDE_CODE_GIT_BASH_PATH 지정
설치 스크립트 TLS 오류	PowerShell의 TLS 설정	[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12 실행 후 재설치
python3: command not found	Windows Python 명령 차이	python --version 또는 py --version 확인

관리자 권한으로 다시 실행하지 않는다

환경 점검표

첫 대화 전에 여덟 항목 확인

해당하지 않는 항목은 문제 해결 장표를 보고 직접 해결한 뒤 다음으로 넘어간다

claude --version에 버전이 출력된다
2.x 버전

claude doctor에 문제가 없다
마지막 줄 확인

claude auth status에 계정이 표시된다
구독 또는 Console 계정

python3 --version이 출력된다
Windows: python · py

git --version이 출력된다
Windows는 Git Bash 포함

~/work/vibe에 트랙 폴더가 있다
네 개 가운데 본인 트랙

트랙 폴더 경로에 공백 · 한글이 없다
pwd로 확인

트랙 폴더에서 claude가 실행된다

입력창 아래 manual mode on

여덟 항목이 모두 해당하면 1-3 첫 대화를 바로 시작한다

실습 · L01

환경 점검과 트랙 폴더 첫 실행

Terminal · L01 점검

```
$ claude --version
$ claude doctor
$ python3 --version
$ cd ~/work/vibe/sales-quote
$ ls
  README.md  data  inbox  report  outbox
$ claude
입력창 아래 manual mode on 확인
› 이 폴더에 무엇이 있는지 알려줘
```

실습 목표

설치 · 인증 · 폴더를 명령으로 확인하고, 본인 트랙 폴더에서 Claude Code를 실행한다

실습 방법

⏱ **예상 소요 10분**

1. claude --version · doctor · auth status를 차례로 실행한다
2. python3 --version과 git --version을 확인한다 (Windows: python · py)
3. cd로 본인 트랙 폴더에 들어가 pwd · ls로 위치를 확인한다
4. claude 실행 → 폴더 신뢰 승인 → Shift+Tab으로 manual mode on
5. '이 폴더에 무엇이 있는지 알려줘'를 입력한다

Pass 기준: Claude Code가 트랙 폴더의 파일 목록을 보고한다

핵심 정리

1-2 정리 · 실행 환경 준비 완료

CLI의 역할
파일을 직접 읽고 저장하고 명령 실행

다섯 층
터미널 → 실행 도구 → Claude Code → 인증 → 폴더

설치 원칙
네이티브 설치 · Windows는 Git 먼저 · sudo 금지

권한 모드
auto로 시작 · 실습은 manual

경로 원칙
~/work/vibe · 영문 · 공백 없음

환경 점검은 아래층에서
위층 순서로 한다

NEXT
1-3 · 첫 대화와 화면 읽기

가져갈 것
문제가 생기면 claude doctor부터 실행.
설치 요약과 문제 해결은 부록 A에 있음.
과정 중에는 버전을 변경하지 않음.

1-3

첫 대화와 화면 읽기

화면에 나오는 줄을 읽을 수 있어야 맡긴 일을 확인할 수 있다

01
화면 읽기
입력란 · 도구 호출 줄 · 승인 창

02
대화 제어
Esc · /rewind · /clear · @

03
첫 결과물
트랙 자료로 결과물 1건

실습 L02

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

동작 원리

Claude Code는 수집 · 실행 · 확인 반복

이 반복을 에이전틱 루프라고 하며, 각 단계는 화면의 ● 줄로 확인할 수 있다



세 단계는 섞여서 진행되므로 의도와 다르면 Esc로 중단하고 방향을 변경한다

실행 절차

Claude Code는 작업 폴더 안에서 claude로 시작

폴더 밖에서 실행하면 자료를 찾지 못한다

1 · 이동

트랙 폴더로 이동

cd로 트랙 폴더 진입
pwd로 위치 확인
바탕화면에서 실행하지 않기

→ **cd sales-quote**

2 · 실행

claude 입력

입력창이 나타남
아래에 모드 표시
manual mode on 확인

→ **claude**

3 · 지시

문장 입력

한국어 문장 그대로 입력
Enter로 전송
줄바꿈은 Shift+Enter

→ **첫 지시**

4 · 종료

/exit

대화는 자동 저장
claude -c로 이어 가기
claude -r로 선택해 열기

→ **/exit**

data 폴더가 없다는 답이 나오면 pwd부터 확인한다

첫 질문

새 폴더에서는 수정보다 질문이 먼저

질문은 넓은 범위에서 좁은 범위로 진행하며, 읽을 파일은 Claude Code가 스스로 찾는다

순서	질문	얻는 것
① 전체	이 폴더에 무엇이 있는지 알려줘	자료의 종류와 건수
② 형식	data/의 파일은 열 이름이 어떻게 돼?	다룰 수 있는 항목
③ 용어	이 폴더에서 사용하는 용어를 표로 정리해줘	같은 말을 사용하는 기준
④ 한 건	문의 한 건이 초안이 되기까지 따라가 봐	어디서 무엇을 읽는지

질문은 파일을 변경하지 않으며, 첫 수정은 범위가 작은 것 하나로 시작한다

화면 읽기

화면을 읽는 네 가지 줄

무엇을 읽고 무엇을 저장했는지는 ● 줄에 모두 남는다

화면 요소	뜻	확인할 것
› 입력란	지시를 입력하는 곳	보낸 문장이 의도와 같은지
● 도구 호출 줄	Read · Edit · Write · Bash 등 실행한 동작	읽은 파일과 저장한 파일이 맞는지
본문 답변	작업 결과와 설명	숫자 · 날짜 · 파일명
승인 창	파일을 저장하거나 명령을 실행하기 직전의 확인	어느 파일에 무엇을 하는지
아래 표시줄	현재 권한 모드	manual mode on인지

답변보다 ● 줄을 먼저 읽는다. 의도하지 않은 파일을 읽었다면 답도 틀리기 때문이다

승인 창

승인 창에 답하는 세 가지 선택

manual 모드에서 파일을 저장하거나 명령을 실행하기 전에 표시된다

선택	뜻	언제 선택하나
허용	이번 한 번만 실행	처음 보는 명령 · 실제 업무 폴더
이 세션 동안 계속 허용	같은 종류의 동작은 다시 묻지 않음	실습 폴더에서 같은 수정이 반복될 때
거부	실행하지 않음 · 이유를 입력할 수 있음	대상 파일이 다르거나 뜻을 모를 때

거부해도 파일은 손상되지 않으며, 이유를 입력하면 그에 맞춰 다시 시도한다

대화 제어

중단 · 되돌리기 · 비우기부터 익히기

방향이 의도와 다르면 기다리지 말고 바로 중단한다

조작	기능	동작	사용 시점
Esc	중단	진행 중인 작업 중단 · 한 일은 남음	방향이 어긋났을 때
Esc Esc · /rewind	되돌리기	이전 시점으로 대화와 파일을 되돌림	잘못 수정한 파일을 복구할 때
/clear	비우기	대화를 비우고 새로 시작	주제가 변경될 때
/btw	곁가지 질문	하던 작업과 별개로 짧게 질문	용어가 궁금할 때
Shift+Tab	모드 전환	권한 모드를 순서대로 변경	계획만 받고 싶을 때

처음 익힐 조작은 Esc · /clear · Shift+Tab 세 가지이다

세션 종료와 재개

폴더별로 저장되어 다시 열 수 있는 대화

따로 저장하지 않아도 대화는 로컬에 기록된다

입력	동작	비고
/exit · Ctrl+D	Claude Code 종료	대화는 자동으로 저장됨
Ctrl+C	입력이나 진행 중인 응답 취소	한 번으로는 종료되지 않음
claude -c	이 폴더의 마지막 대화를 이어서 열기	점심 뒤 · 다음 날
claude -r	이전 대화 목록에서 선택해 열기	며칠 전 작업으로 돌아갈 때
/rename 이름	지금 대화에 이름 추가	목록에서 찾기 쉽게

다시 연 대화에서는 세션 중 허용했던 승인을 다시 받는다

@ 파일 지정

읽을 파일은 @로 지정

지정하지 않으면 어떤 파일을 읽을지 Claude Code가 정한다

지정 없음

"견적 문의에 답장 써줘"

어느 문의인지, 단가는 어디 있는지 없음

1. **읽는 파일**
   폴더를 탐색해 스스로 선택
2. **걸리는 시간**
   찾는 단계가 추가됨
3. **결과**
   다른 문의에 답할 수 있음
4. **확인**
   ● 줄을 모두 읽어야 함

→ 결과를 믿기 어려움

@ 지정

"@inbox/견적문의_01.txt"

"@data/단가표.csv 기준으로 답장 써줘"

1. **읽는 파일**
   지정한 두 파일
2. **걸리는 시간**
   바로 읽음
3. **결과**
   지정한 문의에 답함
4. **확인**
   ● Read 줄이 두 건인지

→ 읽은 범위가 분명하다

@ 뒤에 경로를 입력하고 Tab으로 완성한다

입력 방법

글 말고도 세 가지 방법으로 더 입력

오류 화면은 글로 옮기지 않고 캡처를 그대로 붙여 넣는다

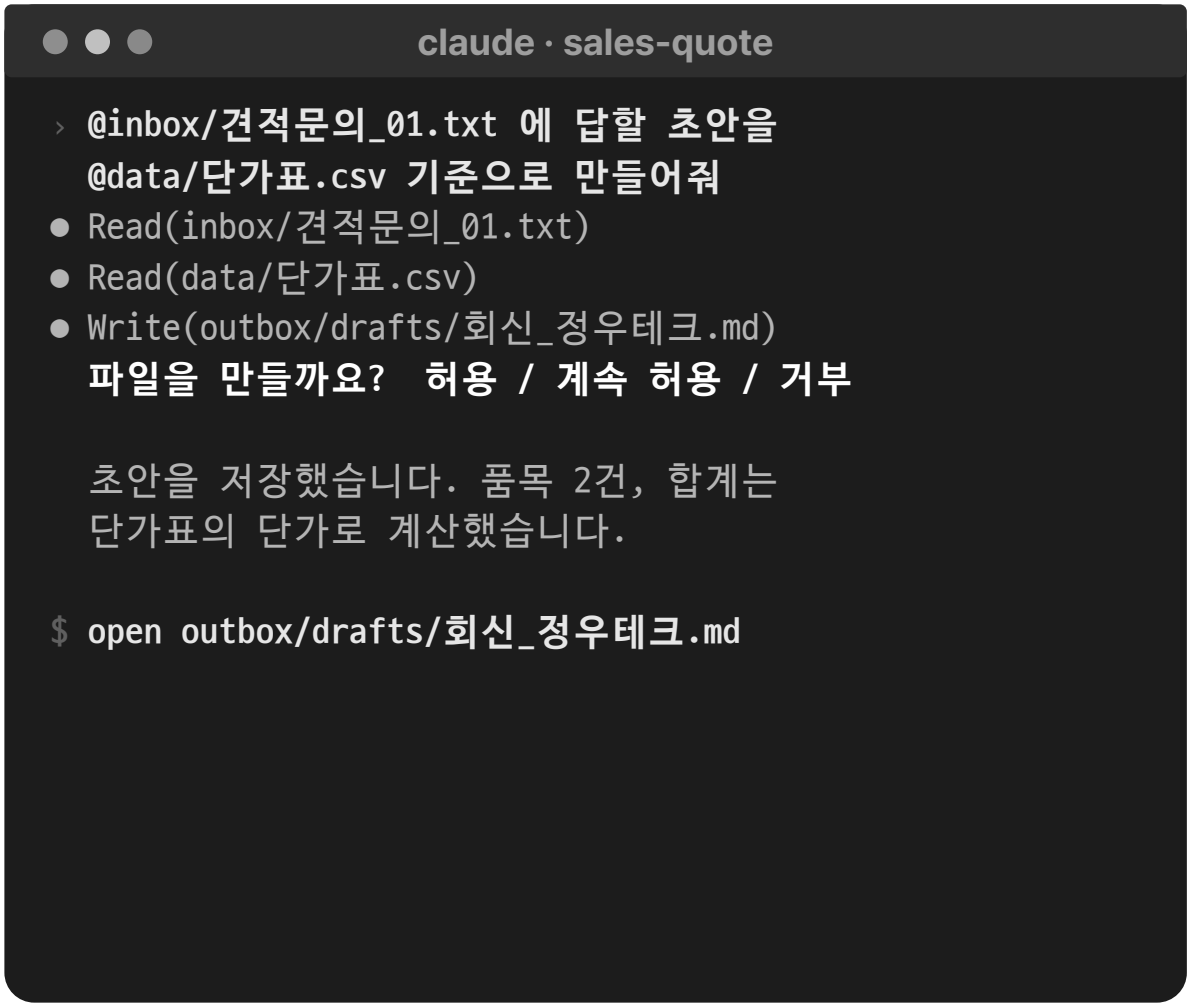
방법	기능	입력 예	사용 시점
@경로	파일 · 폴더 지정	@data/단가표.csv	읽을 자료를 정할 때
이미지 붙여넣기	화면 캡처를 함께 보냄	Ctrl + V	오류 화면 · 양식 사진
! 명령	셸 명령을 바로 실행	! ls report	결과 파일을 확인할 때
Shift+Enter	여러 줄로 나눠 입력	네 요소를 줄마다	지시문이 길 때

! 로 실행한 명령과 그 출력은 대화에 함께 남는다

따라 하기 · 첫 결과물

지시 한 문장으로 첫 결과물 만들기

예시 화면이며, 출력 문구는 버전과 자료에 따라 다르다



① 지시
문의 파일과 단가표를 @로 지정

② 읽기
● Read 두 건을 확인

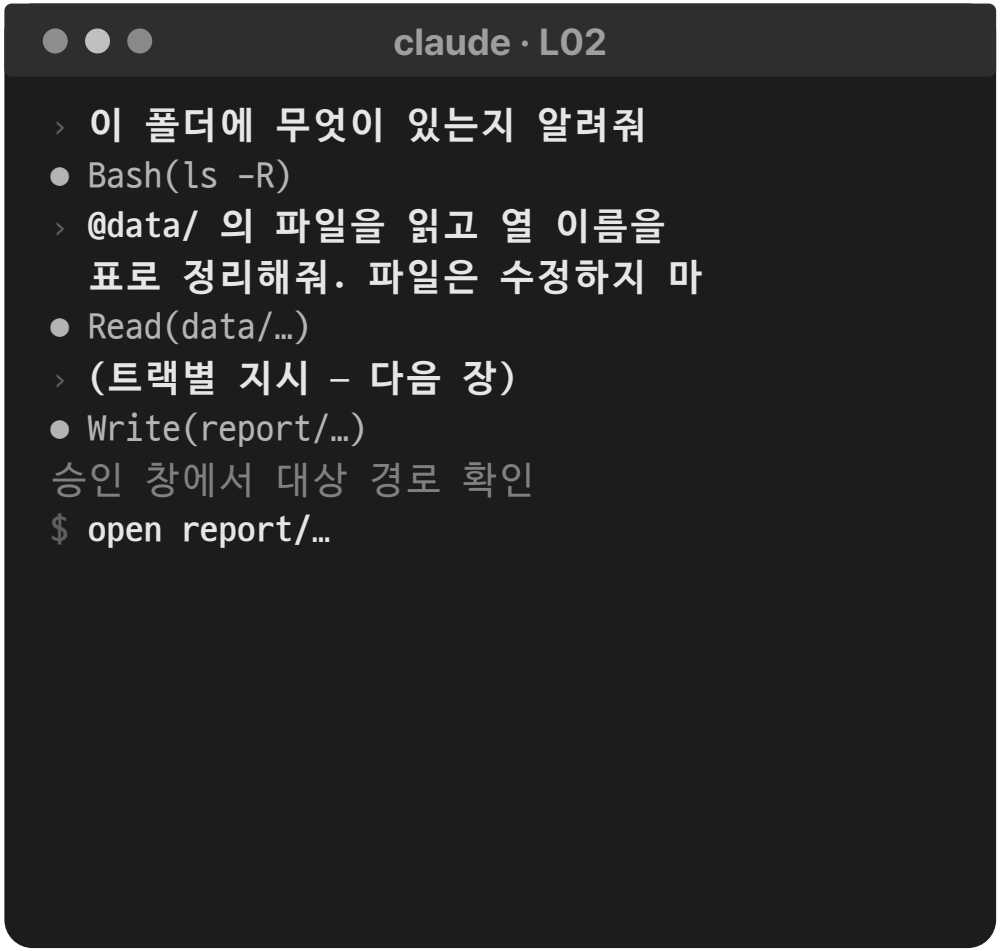
③ 승인 창
저장 위치가 outbox/drafts/인지

④ 답변
품목 수와 합계를 읽음

⑤ 열어 보기
파일을 직접 열어 확인

실습 · L02

트랙 자료로 첫 결과물 만들기



실습 목표

본인 트랙 자료를 읽히고, 지시 한 문장으로 결과물 한 건을 만든다. 화면의 ● 줄을 읽어 무엇을 읽고 저장했는지 기록한다

실습 방법

⏱ **예상 소요 15분**

1. 트랙 폴더에서 claude를 실행하고 manual mode on을 확인한다
2. data/의 파일을 @로 지정해 열 이름을 표로 받는다
3. 다음 장의 트랙별 지시 문장을 입력한다
4. 승인 창에서 저장 위치를 확인하고 허용한다
5. 만들어진 파일을 열어 ● 줄과 함께 확인한다

Pass 기준: 결과 파일 1건이 생기고 data/는 변경되지 않았다

L02 · 트랙별 과제

트랙마다 다른 읽을 자료와 첫 지시

지시 문장은 의도적으로 짧게 구성했으며, 1-4에서 수정한다

트랙	읽을 자료	첫 지시	결과 파일
A · 영업	inbox/견적문의_01.txt data/단가표.csv	이 문의에 답할 견적 회신 초안을 만들어줘	outbox/drafts/회신_거래처.md
B · 인사	data/직원명단.csv data/수료기록.csv · 교육과정.csv	교육을 아직 듣지 않은 사람을 정리해줘	report/교육이수.md
C · 재무 · 구매	data/발주.csv data/세금계산서.csv	두 파일을 대조해 다른 건을 찾아줘	report/대조.csv
D · 품질 · IT 운영	data/불량보고/	이번 주 불량 보고를 요약해줘	report/품질주간.md

결과가 만족스럽지 않아도 삭제하지 않고 1-4에서 비교한다

자주 막히는 곳

첫 대화에서 막히는 네 가지

Q **'data 폴더가 없습니다'라는 답이 나와요**

claude를 다른 폴더에서 실행한 경우이다. /exit로 나가 cd로 트랙 폴더에 들어간 뒤 다시 실행한다.
pwd 결과의 끝이 트랙 폴더 이름인지 먼저 확인한다.

Q **승인 창이 나오지 않고 바로 파일이 만들어져요**

auto 모드나 accept edits 모드인 경우이다. 입력창 아래 표시를 확인한다.
Shift+Tab으로 manual mode on으로 변경한다.

Q **승인 창에서 거부를 눌렀어요**

손상된 파일은 없다. '다시 만들어줘'라고 입력하면 승인 창이 다시 나온다.
거부할 때 이유를 입력하면 그 이유를 반영해 다시 시도한다.

Q **만들어진 파일이 열리지 않아요**

macOS는 open 파일경로, Windows Git Bash는 start 파일경로를 입력한다.
Finder나 탐색기에서 report 폴더를 열어 더블클릭해도 된다.

핵심 정리

1-3 정리 · 화면을 읽고 중단하는 방법

실행
트랙 폴더 안에서 claude

화면
● 줄에서 읽은 파일과 저장한 파일을 확인

승인 창
대상 경로와 명령을 보고 답함

제어
Esc · /rewind · /clear

지정
읽을 파일은 @로 지정

답변보다 ● 줄을 먼저 읽는다

NEXT
1-4 · 지시문 작성

가져갈 것

방향이 의도와 다르면 바로 Esc를 누름.
주제가 변경되면 /clear로 새로 시작.
L02 결과 파일은 지우지 않고 둠.

1-4
지시문 작성

같은 자료로도 지시에 따라 결과가 달라진다. 지시를 문장 네 줄로 고정한다

01
네 요소
역할 · 맥락 · 형식 · 제약

02
네 기법
예시 · 단계 · 환각 방지 · 되묻기

03
CLAUDE.md
반복 지시를 파일로

실습 L03

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

지시의 차이

결과를 운에 맡기는 막연한 지시

L02의 지시와 네 요소를 넣은 지시를 비교한다

L02

막연한 지시

"이 문의에 답할 초안을 만들어줘"

1. **형식**
   실행할 때마다 달라짐
2. **없는 품목**
   비슷한 단가로 채울 수 있음
3. **할인 · 납기**
   임의로 넣을 수 있음
4. **확인**
   전체를 다시 읽어야 함

→ 매번 수동으로 수정

→

L03

네 요소 지시

역할 · 맥락 · 형식 · 제약 작성

1. **형식**
   인사 → 표 → 납기 순서로 고정
2. **없는 품목**
   '확인 필요'로 표시
3. **할인 · 납기**
   적용하지 않는다고 작성
4. **확인**
   표와 '확인 필요'만 확인

→ 볼 곳이 정해짐

지시문이 길어지는 것은 비용이 아니며, 그만큼 수정하는 시간이 줄어든다

네 요소

지시문은 역할 · 맥락 · 형식 · 제약 네 줄로 작성

빠진 요소는 Claude Code가 스스로 채운다

요소	작성할 내용	트랙 A 예
역할	누구의 입장에서 작성하는지	너는 영업지원 담당자야
맥락	무엇을 위한 것이고 자료는 어디에 있는지	inbox/의 문의에 회신할 초안이야. 단가는 data/단가표.csv에 있어
형식	결과의 구성 · 순서 · 저장 위치	인사 → 품목 · 수량 · 단가 · 합계 표 → 납기 안내. outbox/drafts/에 저장
제약	하지 말 것과 모를 때의 처리	단가표에 없는 품목은 '확인 필요'. 할인은 적용하지 마

네 줄 가운데 가장 자주 빠지는 것은 제약이다

네 기법

결과가 흔들릴 때 더하는 기법 네 가지

네 요소로 부족할 때 한 줄씩 추가한다

기법	사용 방법	효과
예시	원하는 출력 한 건을 그대로 보여 줌	형식 설명보다 정확하게 전달됨
단계	읽기 → 계산 → 작성처럼 순서를 나눔	중간 결과를 확인할 수 있음
환각 방지	자료에 없으면 '확인 필요'로 표기하게 함	없는 값을 만들지 않음
되묻기	애매한 점은 시작 전에 물어보게 함	잘못된 가정으로 끝까지 가지 않음

환각 방지와 되묻기 두 줄은 모든 지시에 넣는다

지시문 예 · 트랙 A

견적 회신 초안 지시문

네 요소에 환각 방지와 되묻기를 더해 여섯 줄로 구성한다

지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 영업지원 담당자야.

맥락: @inbox/견적문의_01.txt 에 회신할 초안이 필요해. 단가는 @data/단가표.csv 에 있어.

형식: 인사 → 품목 · 수량 · 단가 · 합계 표 → 납기 안내 순서로 outbox/drafts/에 저장해줘.

제약: 단가표에 없는 품목은 '확인 필요'로 적고 금액을 만들지 마. 할인은 적용하지 마.

되묻기: 시작 전에 애매한 것이 있으면 먼저 물어봐.

입력

inbox/견적문의_01.txt · data/단가표.csv

출력

outbox/drafts/회신_거래처.md

Pass ①

초안의 단가 = 단가표의 단가

Pass ②

단가표에 없는 품목은 '확인 필요'

협업 경계선

할인 · 납기 확정과 발송은 담당자

적용 기법

환각 방지 · 되묻기

지시문 옆에 Pass 기준 두 줄을 함께 작성해 둔다

지시 수정하기

한 번에 하나씩, 무엇이 틀렸는지 밝혀 수정

'다시 해줘'라는 지시는 같은 결과를 다시 만들 뿐이다

막연한 요청	수정한 요청	달라지는 점
좀 더 깔끔하게 해줘	표의 열을 품목 · 수량 · 단가 · 합계 네 개로 줄여줘	무엇을 변경할지 정해짐
틀렸어, 다시 해줘	2행 합계가 단가×수량과 달라. 계산식을 같이 적어줘	틀린 위치와 기대값이 전달됨
전체적으로 손봐줘	인사말만 두 문장으로 줄여줘. 표는 그대로 둬	수정하지 않을 범위가 정해짐
알아서 잘 해줘	애매한 점이 있으면 먼저 질문해줘	가정 대신 질문이 돌아옴

수정할 대상 · 기대한 값 · 실제 값을 한 문장에 작성한다

CLAUDE.md

반복하는 지시는 CLAUDE.md에 작성

폴더 맨 위의 CLAUDE.md는 새 대화마다 자동으로 먼저 읽힌다

sales-quote/CLAUDE.md

```
# 견적 회신 도우미
## 역할
- 너는 영업지원 담당자다.
## 자료
- 단가: data/단가표.csv (수정 금지)
- 문의: inbox/
## 형식
- 인사 → 표(품목·수량·단가·합계) → 납기 안내
- 저장: outbox/drafts/회신_거래처.md
## 제약
- 단가표에 없는 품목은 '확인 필요'
- 할인 금지 · 발송 금지 · 기준일 2026-06-15
```

역할 · 자료

매번 같은 내용은 여기에 둠

형식

저장 위치까지 작성

제약

금지와 모를 때의 처리

기준일

날짜 판단이 있는 업무는 반드시

짧게 유지

길어지면 지켜지지 않는 줄이 생김

CLAUDE.md 다루기

CLAUDE.md는 명령으로 만들고 수정

직접 작성해도 되지만 초안은 Claude Code에 맡긴다

방법	입력	하는 일	사용 시점
초안 만들기	/init	폴더를 읽고 CLAUDE.md 초안 생성	처음 한 번
대화로 수정하기	"CLAUDE.md의 제약에 한 줄 추가해줘"	지시한 줄만 수정	규칙이 늘어날 때
직접 열기	/memory	적용 중인 지침 파일을 열어 편집	내용을 확인할 때
개인 공통	~/.claude/CLAUDE.md	모든 폴더에 적용되는 개인 지침	말투 · 언어처럼 공통인 것

CLAUDE.md를 수정한 뒤에는 /clear로 새 대화에서 확인한다

작성 기준

CLAUDE.md에 작성할 줄과 작성하지 않을 줄

파일이 길어질수록 지켜지지 않는 줄이 늘어나므로 200줄 이내로 유지한다

구분	작성할 것	작성하지 않을 것
사실	자료만 읽어서는 알 수 없는 기준일 · 단위 · 저장 위치	자료를 읽으면 알 수 있는 내용
형식	기본과 다르게 받고 싶은 형식 규칙	'정확하게 작성한다' 같은 당연한 말
금지	수정하면 안 되는 폴더 · 하면 안 되는 약속	이번 한 번만 필요한 지시
변동	드물게 변경되는 업무 규칙	자주 변경되는 값 · 긴 설명과 사례

줄마다 '이 줄이 없으면 실수하는가'를 검토하고, 그렇지 않으면 삭제한다

실습 · L03

지시문을 수정하고 CLAUDE.md로 옮기기

```
claude · L03

› (네 요소 지시문 — 부록 B)
● Read(…)  ● Write(…)
L02 결과와 나란히 열어 비교
› 지금 지시의 역할·형식·제약을
  CLAUDE.md로 옮겨줘
● Write(CLAUDE.md)
› /clear
› @inbox/… 에 답할 초안을 만들어줘
짧은 지시로 같은 형식이 나오는지 확인
```

실습 목표

L02와 같은 자료로 네 요소 지시문을 입력해 결과를 비교하고, 반복되는 부분을 CLAUDE.md로 옮긴다

실습 방법

⏱ **예상 소요 20분**

1. 부록 B의 본인 트랙 지시문을 입력해 결과물을 다시 만든다
2. L02 결과와 나란히 열어 달라진 점 세 가지를 작성한다
3. 역할 · 자료 · 형식 · 제약을 CLAUDE.md로 옮기게 한다
4. /clear 후 짧은 지시 한 줄로 다시 실행한다
5. CLAUDE.md의 형식과 제약이 지켜졌는지 확인한다

Pass 기준: /clear 뒤 짧은 지시로도 같은 형식의 결과가 나온다

L03 · 트랙별 과제

트랙마다 다른 형식과 제약의 내용

지시문 전체는 부록 B에 수록했다

트랙	형식	제약	달라져야 할 점
A · 영업	인사 → 표 → 납기 안내	단가표에 없는 품목은 '확인 필요' · 할인 금지	없는 품목에 금액이 없음
B · 인사	부서별 이수율 표 → 미이수자 목록	기한이 지난 사람만 '지연' · 명단에 없으면 '확인 필요'	'지연'이 기준일로 판정됨
C · 재무 · 구매	발주번호 · 두 금액 · 차액 · 판정	판정은 일치 · 금액 차이 · 계산서 없음 세 가지만	판정 값이 세 가지로 고정됨
D · 품질 · IT 운영	요약 3줄 → 유형별 표 → 확인 필요	원인이 없으면 '원인 미기재' · 추정 금지	없는 원인이 채워지지 않음

네 트랙 모두 제약 한 줄이 결과를 가장 크게 변경한다

핵심 정리

1-4 정리 · 지시를 문장으로 고정

네 요소
역할 · 맥락 · 형식 · 제약

네 기법
예시 · 단계 · 환각 방지 · 되묻기

수정하기
한 번에 하나 · 대상과 기대값 작성

CLAUDE.md
반복 지시는 파일로 옮김

확인
/clear 뒤 새 대화에서 다시 실행

제약 한 줄이
결과를 가장 크게 변경한다

NEXT
1-5 · 검증과 오류 해결

가져갈 것

지시문은 부록 B의 템플릿에서 시작.
CLAUDE.md는 짧게 유지.
기준일이 필요한 업무는 반드시 작성.

1-5

검증과 오류 해결

결과가 맞는지 판단하는 기준과, 틀렸을 때 수정하는 절차

01
Pass 기준
지시 전에 두 문장

02
검증 네 가지
출처 · 재계산 · 반례 · 새 대화

03
오류 해결
원문 전달 → 원인 → 수정 → 재실행

실습 L04

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Pass 기준

Pass 기준은 지시하기 전에 작성

결과를 본 뒤에 기준을 정하면 결과에 기준을 맞추게 된다

나중에

결과를 보고 판단

"대체로 맞는 것 같다"

1. **기준**
   읽는 사람의 인상
2. **빠진 건**
   발견하기 어려움
3. **재실행**
   같은지 비교할 수 없음
4. **완료 시점**
   정할 수 없음

→ 검토가 끝나지 않음

→

먼저

기준을 작성하고 지시

"행 수 = 발주 건수"

1. **기준**
   세어 볼 수 있는 문장
2. **빠진 건**
   건수가 맞지 않아 드러남
3. **재실행**
   같은 기준으로 다시 확인
4. **완료 시점**
   기준을 통과한 때

→ 확인이 몇 분으로 줄어듦

Pass 기준은 건수 · 합계 · 값의 종류처럼 셀 수 있는 것으로 작성한다

검증 수단

확인할 수단을 지시와 함께 전달

수단이 없으면 결과를 확인하는 것은 사람뿐이다

수단	지시에 넣는 문장	Claude Code가 하는 일
기대 건수	결과의 행 수는 문의 건수와 같아야 해	건수를 세어 맞는지 확인
예시 한 건	이 한 건처럼 작성해줘	예시와 형식을 맞춤
재계산	합계 옆에 계산식을 함께 작성해줘	식과 값 비교
다시 실행	끝나면 한 번 더 실행해 결과를 비교해줘	두 결과가 같은지 봄

Pass 기준을 지시문 안에 작성하면 Claude Code가 먼저 확인하고 보고한다

검증 네 가지

결과는 네 가지 방법으로 확인

네 가지 모두 대화로 요청할 수 있으나, 최종 판단은 사람이 한다

방법	요청 문장	잡아내는 것
출처 확인	각 값이 어느 파일 몇 행에서 왔는지 적어줘	자료에 없는 값
재계산	합계마다 계산식을 같이 적어줘	계산 오류 · 단위 혼동
반례 찾기	이 결과가 틀렸다면 어디일지 세 곳을 알려줘	빠뜨린 행 · 경계 조건
새 대화에서 재실행	/clear 뒤 같은 지시를 다시 실행	우연히 맞은 결과

'확실해?'라고 질문하는 대신 확인할 수 있는 근거를 요구한다

트랙별 함정

트랙마다 실습 자료에 들어 있는 함정

함정은 실제 업무 자료에서 흔히 나타나는 형태로 구성했다

트랙	함정 ①	함정 ②	드러나는 Pass 기준
A · 영업	단가표에 없는 품목이 든 문의	수량이 '박스' 단위로 작성된 문의	없는 품목 · 단위는 '확인 필요'
B · 인사	명단에 없는 사번의 수료 기록	기한 뒤에 수료한 기록	이수 + 미이수 = 명단 인원
C · 재무 · 구매	세금계산서가 없는 발주	부가세만큼 금액이 다른 건	대조표 행 수 = 발주 건수
D · 품질 · IT 운영	원인이 적히지 않은 보고	같은 유형의 다른 표기	유형별 건수 합 = 보고 건수

함정에 걸렸다면 지시문에 제약 한 줄이 빠진 것이다

오류 해결 절차

오류는 네 단계로 해결

에러 메시지는 요약하지 않고 원문 그대로 전달한다

STEP 1

원문 전달

에러 메시지를 그대로 복사
직전에 한 일을 한 줄로
추측은 작성하지 않음

→ 원문 그대로

➡

STEP 2

원인 먼저

수정하기 전에 원인을 설명
한 문장으로 요청
이해되지 않으면 다시 질문

→ 원인 한 줄

➡

STEP 3

한 곳만 수정

원인에 해당하는 곳만
다른 부분은 그대로
승인 창에서 범위 확인

→ 변경 1건

➡

STEP 4

다시 실행

같은 명령을 다시 실행
Pass 기준으로 확인
안 되면 STEP 1로

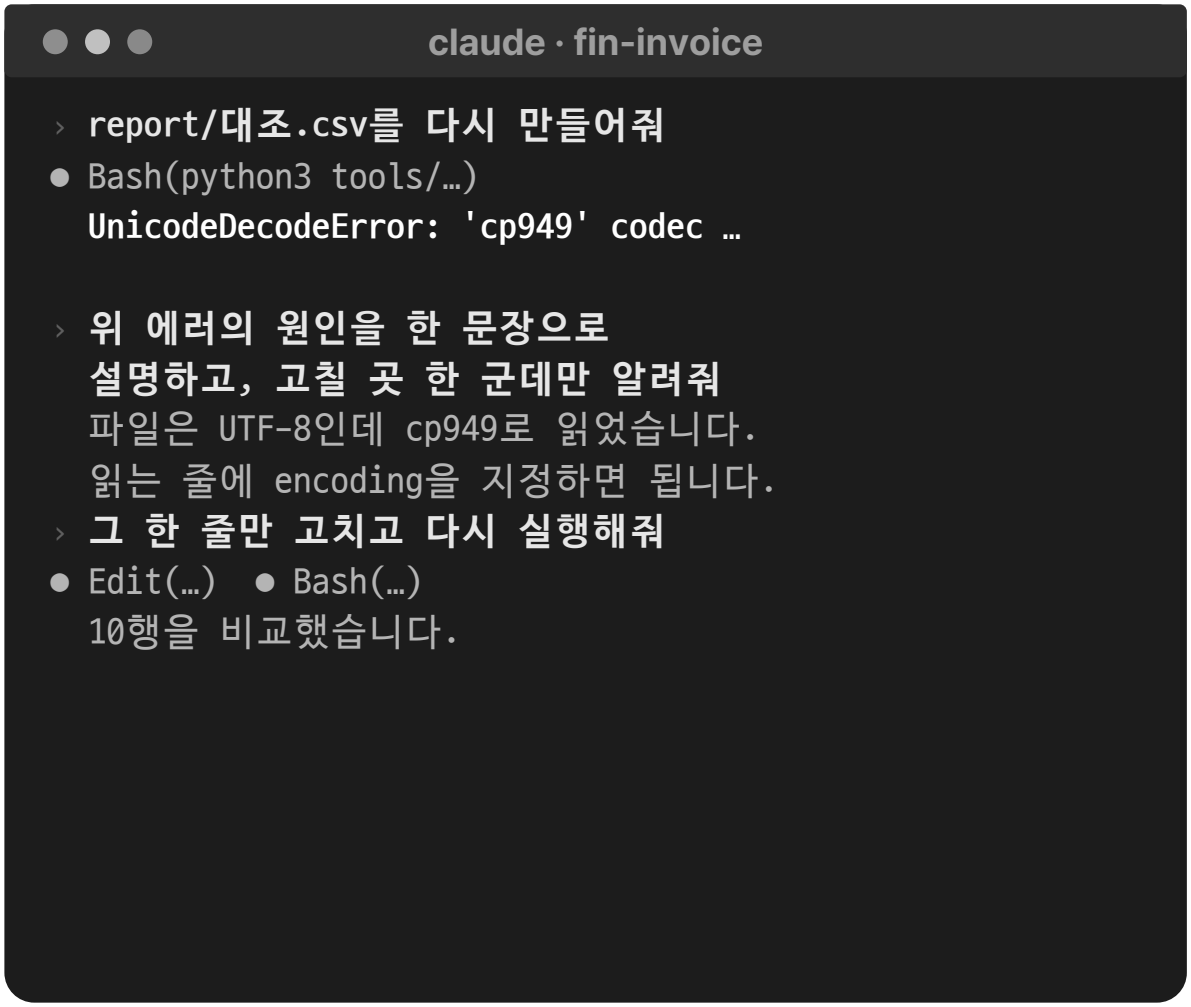
→ 통과 확인

세 번 수정해도 안 되면 /rewind로 되돌리고 지시를 다시 작성한다

따라 하기 · 오류 한 건

에러 원문을 주고 원인부터 질문

예시 화면이며, 출력 문구는 버전과 자료에 따라 다르다



① 에러 발생
메시지를 지우지 않음

② 원인 질문
수정하기 전에 설명부터

③ 범위 지정
한 군데만 수정하게 함

④ 재실행
같은 명령으로 확인

⑤ 판정
행 수를 Pass 기준과 비교

되돌리기

잘못 수정한 것을 되돌리는 방법

되돌릴 수 있는 범위를 알아야 맡길 수 있다

방법	되돌리는 범위	한계
Esc Esc · /rewind	선택한 시점 이후의 대화와 파일 수정	Bash 명령이 변경한 것은 되돌리지 못함
거부 후 다시 지시	아직 실행하지 않은 동작	이미 허용한 것은 해당 없음
원본 다시 복사	data/를 잘못 수정한 경우	원본을 따로 보관해야 함
되돌릴 수 없는 것	발송한 메일 · 외부 시스템 변경	그래서 발송은 사람이 함

되돌릴 수 없는 일은 맡기지 않으며, 맡기더라도 초안까지로 제한한다

/rewind 메뉴

되돌리기 메뉴의 다섯 가지 선택

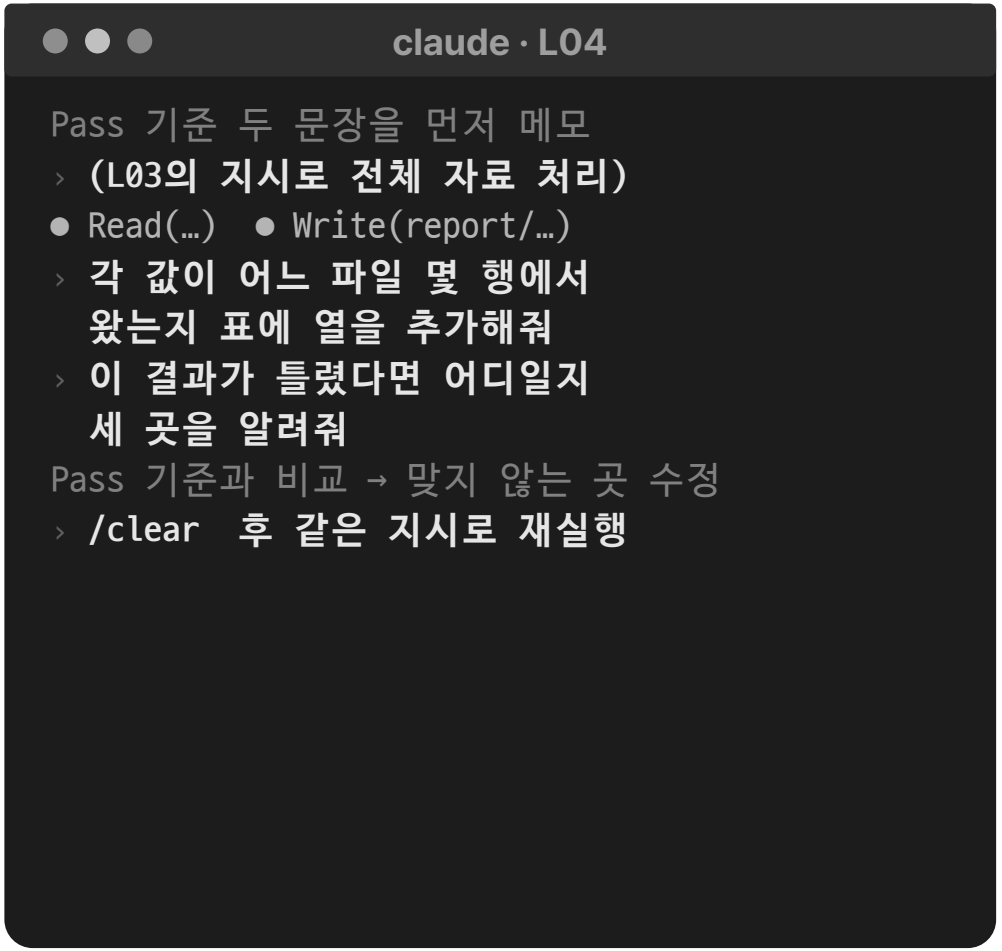
Esc를 두 번 누르거나 /rewind를 입력하면 시점 목록이 열린다

선택	동작	사용 시점
Restore code and conversation	파일과 대화를 그 시점으로 되돌림	지시부터 잘못됐을 때
Restore conversation	대화만 되돌리고 파일은 그대로 둠	결과는 맞고 대화만 길어졌을 때
Restore code	파일만 되돌리고 대화는 남김	수정은 취소하고 논의는 이어 갈 때
Summarize from here	그 시점 이후의 대화를 요약으로 줄임	Context를 비우고 싶을 때
Never mind	아무것도 변경하지 않고 닫음	잘못 열었을 때

되돌리면 그 시점의 지시문이 입력창에 다시 채워져 수정해서 보낼 수 있다

실습 · L04

Pass 기준으로 확인하고 함정 찾기



실습 목표

Pass 기준 두 문장을 먼저 작성하고, 트랙 전체 자료를 처리한 결과를 네 가지 방법으로 검증한다

실습 방법

⏱ **예상 소요 25분**

1. 본인 트랙의 Pass 기준 두 문장을 메모장에 작성한다
2. L03의 지시로 트랙의 전체 자료를 처리한다
3. 출처 확인과 반례 찾기를 요청해 맞지 않는 곳을 찾는다
4. 맞지 않는 곳은 한 번에 한 건씩 대상과 기대값을 밝혀 수정한다
5. /clear 뒤 다시 실행해 Pass 기준을 통과하는지 확인한다

Pass 기준: 함정 두 개가 결과에 표시되어 있다

자주 막히는 곳

검증과 오류에서 막히는 네 가지

Q **수정할수록 다른 곳이 틀어져요**

한 번에 여러 건을 수정하게 한 경우이다. /rewind로 되돌린 뒤 한 건씩 요청한다.
'표는 그대로 둬'처럼 수정하지 않을 범위를 함께 작성한다.

Q **한글이 깨져서 나와요**

파일 인코딩이 맞지 않는 경우이다. 에러 원문이나 깨진 화면을 그대로 보여 주고 원인을 묻는다.
엑셀에서 저장한 CSV는 cp949인 경우가 있다.

Q **새 대화에서는 결과가 달라져요**

앞 대화에서 말로만 준 조건이 CLAUDE.md에 없는 경우이다.
달라진 부분을 찾아 CLAUDE.md의 형식이나 제약에 한 줄로 옮긴다.

Q **'확인 필요'가 너무 많이 나와요**

자료 위치를 지시에 작성하지 않은 경우이다. 맥락에 파일 경로를 작성한다.
그래도 남는 것은 실제로 자료에 없는 값이다. 사람이 채운다.

핵심 정리

1-5 정리 · 기준으로 확인

Pass 기준
지시 전에 셀 수 있는 문장으로

검증
출처 · 재계산 · 반례 · 새 대화

오류 해결
원문 → 원인 → 한 곳 수정 → 재실행

되돌리기
/rewind · 되돌릴 수 없는 일은 맡기지 않음

함정
제약 한 줄이 빠진 자리

근거를 요구하고
건수로 확인한다

NEXT
1-6 · 업무 도구 1호

가져갈 것

숫자 · 날짜 · 이름을 먼저 비교.
세 번 수정해도 안 되면 되돌림.
찾은 함정은 CLAUDE.md의 제약에 작성.

1-6

업무 도구 1호

오늘 만든 지시와 규칙을 커맨드 하나로 묶는다

01
커맨드
.claude/commands/이름.md

02
역할 분담
CLAUDE.md는 규칙, 커맨드는 절차

03
자가 점검
새 대화에서 다시 실행해 보기

실습 L05

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

커맨드 파일

자주 하는 절차는 커맨드 파일로 정리

.claude/commands/quote.md를 만들면 /quote로 실행된다

.claude/commands/quote.md

```
---
description: 견적 문의에 회신 초안을 만든다
---

1. inbox/에서 회신하지 않은 문의를 찾는다.
2. data/단가표.csv에서 품목별 단가를 찾는다.
3. CLAUDE.md의 형식으로 초안을 작성한다.
4. outbox/drafts/에 저장한다.
5. 처리한 문의 수와 '확인 필요' 품목을 보고한다.

대상 문의: $ARGUMENTS
```

파일 이름이 명령 이름

quote.md → /quote

description

목록에 표시되는 한 줄 설명

본문

절차를 번호로 작성

$ARGUMENTS

/quote 뒤에 적은 말이 들어감

규칙은 작성하지 않는다

CLAUDE.md를 따르게 둠

역할 분담

규칙은 CLAUDE.md에, 절차는 커맨드에

같은 내용을 두 곳에 작성하면 한쪽만 수정하게 된다

CLAUDE.md

항상 적용

새 대화마다 자동으로 읽힘

1. **담는 것**
   역할 · 자료 위치 · 형식 · 제약
2. **적용**
   모든 지시에
3. **길이**
   짧게 유지
4. **수정하는 때**
   함정을 새로 찾았을 때

→ 무엇을 지킬지

커맨드

부를 때만

/이름으로 호출할 때 읽힘

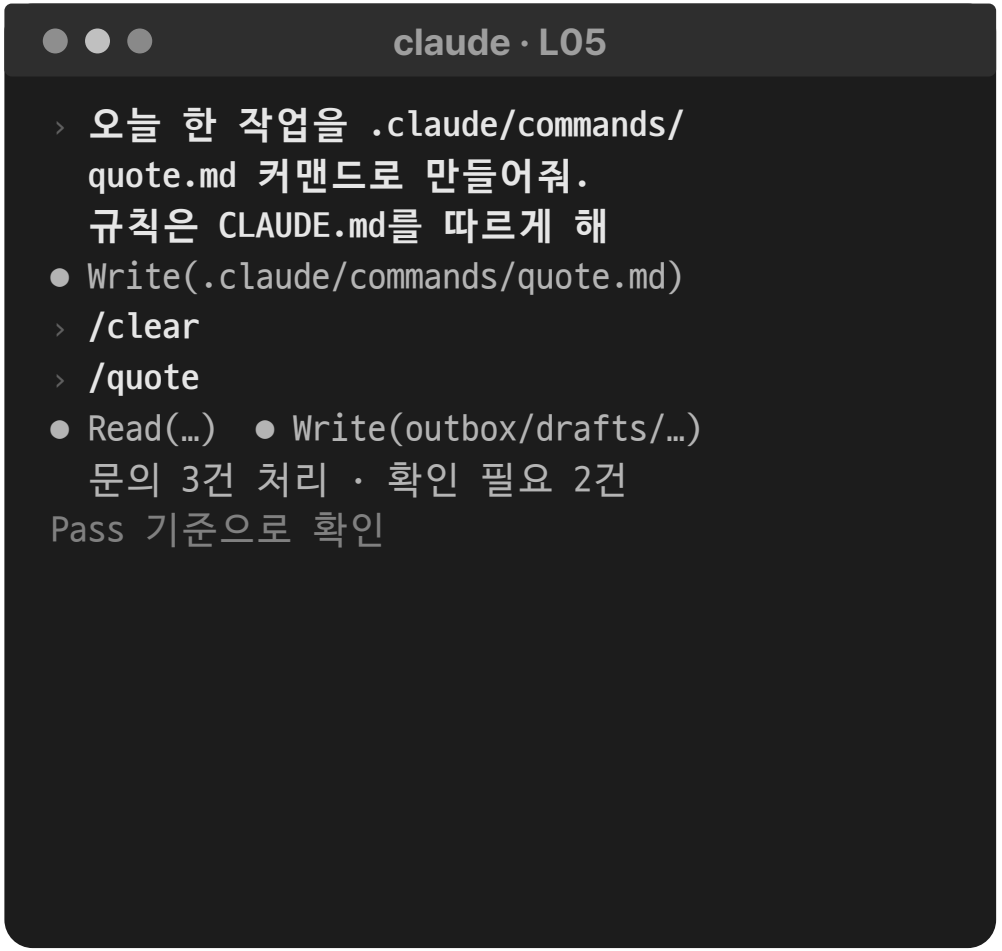
1. **담는 것**
   작업 순서와 보고 항목
2. **적용**
   그 업무를 할 때만
3. **개수**
   업무마다 하나씩
4. **수정하는 때**
   절차가 변경되었을 때

→ 어떤 순서로 할지

커맨드에는 '규칙은 CLAUDE.md를 따른다'고만 작성한다

실습 · L05

커맨드로 업무 도구 1호 만들기



실습 목표

L03 · L04에서 다듬은 지시를 커맨드 파일로 만들고, 새 대화에서 한 줄로 실행해 같은 결과를 얻는다

실습 방법

⏱️ 예상 소요 20분

1. 오늘 작업을 본인 트랙의 커맨드 파일로 만들게 한다
2. 커맨드 파일을 열어 절차가 번호로 적혔는지 확인한다
3. /clear 뒤 커맨드 한 줄로 실행한다
4. L04의 Pass 기준으로 결과를 확인한다
5. 틀린 곳은 CLAUDE.md나 커맨드 중 한 곳만 수정한다

Pass 기준: 새 대화에서 커맨드 한 줄로 Pass 기준을 통과한다

L05 · 트랙별 과제

트랙마다 커맨드 하나 만들기

커맨드가 보고할 항목이 내일 에이전트의 출발점이 된다

트랙	커맨드	절차	보고 항목
A · 영업	/quote	문의 읽기 → 단가 찾기 → 초안 작성 → 저장	처리 건수 · '확인 필요' 품목
B · 인사	/training	명단과 수료 기록 대조 → 이수율 표 → 미이수자 목록	미이수 인원 · 지연 인원
C · 재무 · 구매	/match	발주와 세금계산서 대조 → 대조표 → 차이 목록	판정별 건수 · 차액 합계
D · 품질 · IT 운영	/weekly	보고서 읽기 → 유형별 집계 → 주간 요약	유형별 건수 · '원인 미기재' 건

보고 항목은 Pass 기준과 같은 숫자를 포함하게 만든다

자가 점검

새 대화에서 커맨드를 다시 실행해 보기

앞 대화의 기억 없이도 실행되어야 업무 도구이다

CLAUDE.md에 네 요소가 있다
역할 · 자료 · 형식 · 제약

기준일이 작성되어 있다
2026-06-15

커맨드 파일이 한 개 있다
.claude/commands/

커맨드에 절차가 번호로 작성되어 있다
네다섯 단계

새 대화에서 커맨드가 실행된다
/clear 뒤 실행

결과가 Pass 기준을 통과한다
건수 · 값의 종류

함정 두 개가 결과에 표시된다
'확인 필요' 등

data/ 원본이 그대로이다

행 수 확인

해당하지 않는 항목은 Day 2 시작 전에 수정한다

핵심 정리

Day 1 정리 · 업무 도구 1호 완성

환경
Claude Code 설치 · manual 모드

대화
● 줄 읽기 · Esc · /clear · @

지시
네 요소 · CLAUDE.md

검증
Pass 기준 · 출처 · 재계산

도구
커맨드 한 줄로 재실행

커맨드 한 줄로
같은 결과가 나온다

NEXT
Day 2 · 2-1 에이전트 설계

가져갈 것

트랙 폴더는 그대로 둠. 내일 이어서 사용.
아침에는 claude -c로 오늘 대화를 이어 갈 수 있음.
본인 업무 후보 하나를 생각해 옴.

DAY 02

도구를 에이전트로 넓히기

Day 2 · 2-1 ~ 2-7. 어제의 커맨드에 도구 · Skill · 기억 · 자동 실행 · 승인 규칙을 더한다

전반
2-1 ~ 2-3 · 설계 · 도구 · Skill · Memory

후반
2-4 ~ 2-7 · 자동 실행 · 승인 · 종합

마감
한 사이클 실행 · 회고

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

오늘의 질문

어디까지 스스로 하게 하고, 어디서 중단하게 할 것인가

Day 2가 끝날 때 본인 트랙 폴더로 이 질문에 답할 수 있으면 된다

전반

스스로 하게 하기

계획을 먼저 받음
계산과 조회는 도구로
업무 지식은 Skill로

후반 ①

혼자 실행되게 하기

claude -p로 한 번에 실행
예약으로 반복
실행 기록 남기기

후반 ②

중단하게 하기

원본은 수정 금지
발송은 승인 뒤에만
규칙은 settings.json으로

Pass 기준: 명령 한 줄로 한 사이클이 실행되고, 발송 전에 중단한다

Day 2의 지도

불편한 신호가 보일 때 하나씩 더하는 기능

문제는 가장 작은 기능으로 해결하며, 모든 기능을 처음부터 사용하지는 않는다

보이는 신호	더하는 기능	두는 곳	세션
같은 규칙을 두 번 이상 틀림	CLAUDE.md	폴더 맨 위	1-4
같은 시작 지시를 매번 입력	커맨드 · Skill	.claude/	1-6 · 2-3
계산과 조회가 실행마다 다름	도구 스크립트	tools/	2-2
밖의 시스템 자료를 복사해 줌	MCP	설정 파일	2-2
곁가지 조사로 대화가 길어짐	서브에이전트	.claude/agents/	2-3
매번 같은 검사를 수동으로 함	Hook	settings.json	2-5
다른 폴더에도 같은 구성이 필요하다	Plugin	묶음 하나	2-7

오늘의 세션은 이 표의 위에서 아래로 진행한다

2-1

에이전트 설계

만들기 전에 계획을 받는다. Plan 모드로 폴더에 무엇을 더할지 정한다

01
도구와 에이전트
한 단계와 여러 단계의 차이

02
여섯 구성
규칙 · 도구 · Skill · 기억 · 자동 · 승인

03
Plan 모드
수정 없이 계획만 받기

실습 L06

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

도구에서 에이전트로

에이전트는 여러 단계를 스스로 이어서 진행

어제의 커맨드가 한 번의 작업이라면 오늘은 한 사이클이다

Day 1

업무 도구

사람이 부르고 결과를 받음

1. **시작** — 사람이 커맨드를 입력
2. **범위** — 지정한 자료 한 묶음
3. **판단** — 절차에 작성된 대로
4. **끝** — 결과 파일 1건

→ 사람이 매번 시작

Day 2

에이전트

새 일을 찾아 끝까지 처리

1. **시작** — 예약 또는 명령 한 줄
2. **범위** — 새로 들어온 것 전부
3. **판단** — 규칙과 Skill로 스스로
4. **끝** — 초안 · 보고 · 승인 요청

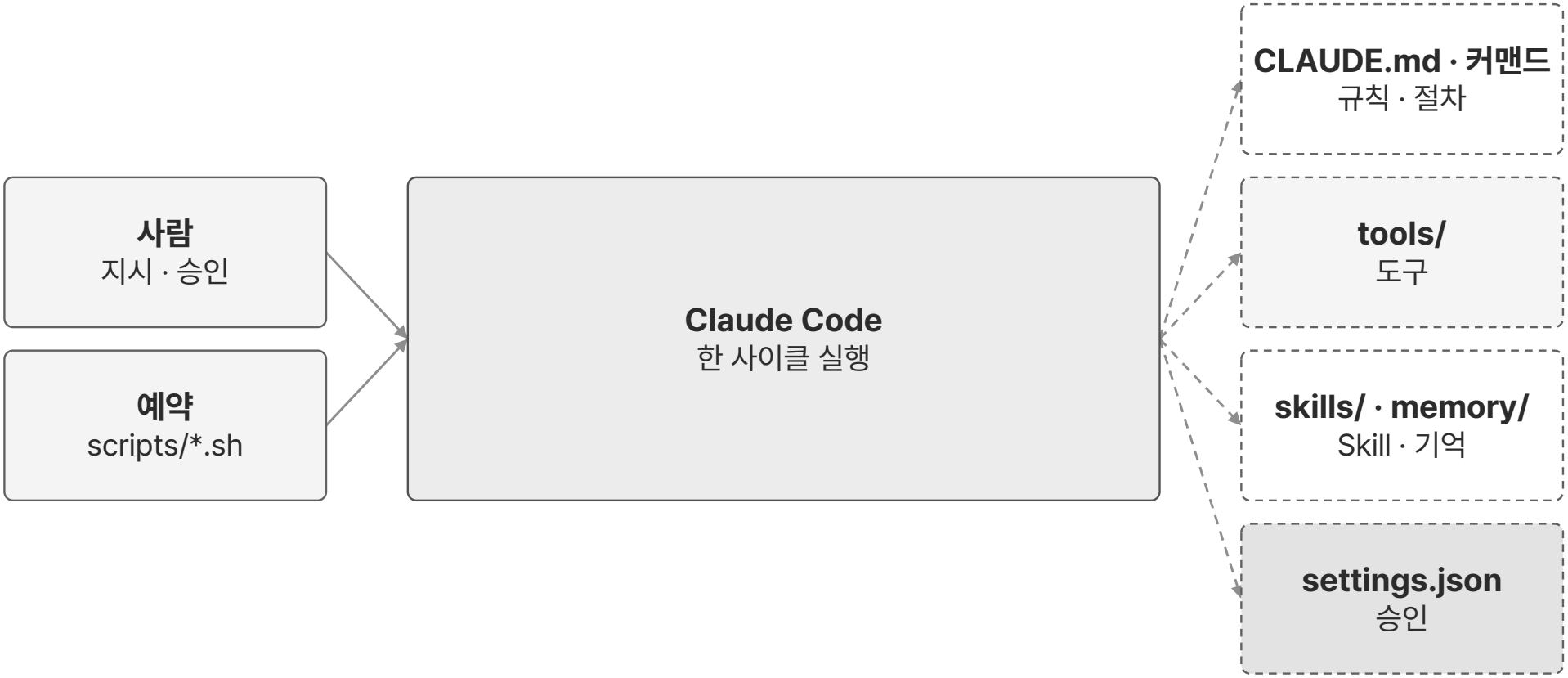
→ 사람은 승인만 함

스스로 하는 범위가 넓어질수록 중단 지점을 먼저 정한다

에이전트의 구성

에이전트는 Claude Code가 폴더의 파일을 읽어 동작

왼쪽은 한 사이클을 시작하는 쪽, 오른쪽은 Claude Code가 읽고 따르는 파일이다



오른쪽 파일을 오늘 세션마다 하나씩 추가한다

여섯 구성

에이전트는 파일 여섯 종류로 구성

여섯 구성은 오늘 세션마다 하나씩 추가한다

구성	파일	하는 일	세션
규칙	CLAUDE.md	항상 지킬 역할 · 형식 · 제약	1-4 (완료)
절차	.claude/commands/	한 사이클의 작업 순서	1-6 · 2-1
도구	tools/*.py	계산 · 조회 · 대조를 정확하게	2-2
Skill · 기억	.claude/skills/ · memory/	업무 지식과 쌓인 사실	2-3
자동 실행	scripts/*.sh	사람 없이 한 사이클 실행	2-4
승인	.claude/settings.json	허용 · 확인 · 금지할 동작	2-5

여섯 가지 모두 폴더 안의 파일이므로 폴더를 복사하면 그대로 이전된다

한 사이클

찾기 · 처리 · 보고 · 중단으로 이루어진 한 사이클

트랙 A의 사이클을 커맨드 파일로 작성한 예이다

.claude/commands/cycle.md

```
---
description: 새 문의를 찾아 초안까지 처리한다
---

1. inbox/에서 처리기록에 없는 문의를 모두 찾는다.
2. 문의마다 /quote의 절차로 초안을 만든다.
3. '확인 필요'가 있는 건은 따로 표시한다.
4. 처리한 문의 이름을 report/처리기록.md에 작성한다.
5. 같은 파일에 건수와 보류 사유를 작성한다.
6. 발송하지 않는다. 승인 요청 목록만 보고한다.
```

찾기

새로 들어온 것을 스스로 찾음

처리

어제 만든 커맨드를 다시 사용

보고

건수와 보류 사유를 파일로

중단

발송 전에 반드시 중단

처리한 것 표시

같은 일을 두 번 하지 않게

Plan 모드

Plan 모드는 파일을 수정하지 않고 계획만 제시

설계 단계에서는 계획을 먼저 받고 승인한 뒤 실행한다

항목	내용	비고
들어가기	Shift+Tab으로 plan mode on까지 전환	한 번만 쓰려면 지시 앞에 /plan
시작할 때 지정	claude --permission-mode plan	설계만 할 때
하는 일	파일을 읽고 탐색해 계획 작성	파일은 수정하지 않음
계획 수정하기	'3번을 둘로 나눠줘'처럼 보완 요청	확정될 때까지 반복
끝내기	계획을 승인하면 실행 단계로 넘어감	승인 전에 범위를 다시 읽음

범위가 넓거나 처음 하는 일은 Plan 모드로 시작한다

작업 순서

큰 작업은 탐색 · 계획 · 실행 · 확인 순서로 진행

곧바로 만들게 하면 의도와 다른 문제를 해결할 수 있다

STEP 1

탐색

관련 파일을 먼저 읽게 함
아직 수정하지 말라고 작성
읽은 내용을 요약받음

→ 현재 상태 파악

➡

STEP 2

계획

Plan 모드에서 단계를 받음
되돌아온 질문에 답함
범위를 좁힘

→ 승인한 계획

➡

STEP 3

실행

계획대로 한 단계씩 진행
승인 창을 읽음
범위 밖 변경은 거부

→ 변경된 파일

➡

STEP 4

확인

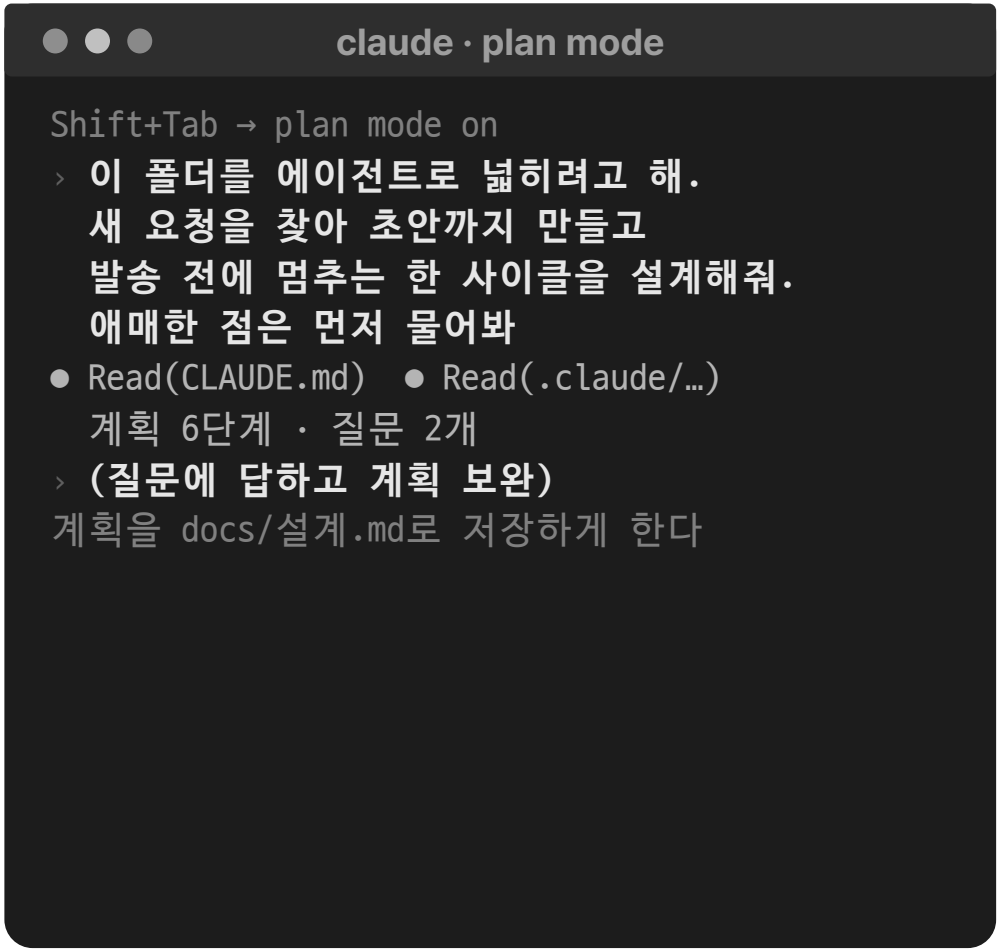
Pass 기준으로 확인
새 대화에서 다시 실행
기록을 남김

→ 통과 확인

한 문장으로 설명되는 작은 수정에는 이 순서가 필요 없다

실습 · L06

Plan 모드로 에이전트 설계하기



실습 목표

Plan 모드에서 본인 트랙의 한 사이클을 설계하게 하고, 질문에 답해 계획을 확정한다

실습 방법

⏱ **예상 소요 15분**

1. Shift+Tab으로 plan mode on을 확인한다
2. 한 사이클의 목표와 중단 지점을 말하고 설계를 요청한다
3. 되돌아온 질문에 답하고 단계를 보완한다
4. 계획을 승인하고 docs/설계.md로 저장하게 한다
5. 설계에 여섯 구성 중 무엇이 들어 있는지 표시한다

Pass 기준: 설계의 마지막 단계가 '발송하지 않고 보고'이다

L06 · 트랙별 과제

트랙마다 다른 한 사이클의 시작과 중단

중단 지점은 어제 정한 협업 경계선과 같다

트랙	새로 찾을 것	스스로 처리할 것	중단 지점
A · 영업	inbox/의 새 견적 문의	단가 조회 · 회신 초안	발송 · 할인 결정
B · 인사	기한이 지난 미이수자	대조 · 독려 메일 초안	발송 · 예외 인정
C · 재무 · 구매	새 세금계산서와 발주	대조표 · 확인 요청 초안	발송 · 차이 처리 결정
D · 품질 · IT 운영	이번 주 불량 보고	집계 · 주간 요약 · 조치 요청 초안	배포 · 원인 판정

중단 지점이 설계에 없으면 승인하지 않는다

핵심 정리

2-1 정리 · 계획 먼저 받기

에이전트
여러 단계를 스스로 연결

여섯 구성
규칙 · 절차 · 도구 · Skill · 자동 · 승인

한 사이클
찾기 · 처리 · 보고 · 중단

Plan 모드
수정 없이 계획만

설계 문서
docs/설계.md

넓게 맡길수록
중단 지점을 먼저 정한다

NEXT
2-2 · 도구 연동

가져갈 것

처음 하는 일은 Plan 모드로 시작.
계획의 질문에 답한 뒤 승인.
설계 문서는 오늘 하루의 기준이 됨.

2-2

도구 연동

계산과 조회는 말로 시키지 않고 도구 스크립트에 맡긴다

01
왜 도구인가
같은 입력에 같은 결과

02
도구 만들기
대화로 스크립트 작성

03
언제 사용하는지
CLAUDE.md에 사용 규칙

실습 L07

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

왜 도구인가

말로 시킨 계산은 실행할 때마다 달라질 가능성

정확해야 하는 부분만 스크립트로 고정한다

말로

매번 새로 계산

"합계를 계산해줘"

1. **방법**
   그때그때 정함
2. **같은 입력**
   결과가 달라질 수 있음
3. **검증**
   매번 다시 계산해야 함
4. **자료가 많으면**
   일부를 건너뛸 수 있음

→ 건수가 늘면 불안하다

→

도구로

스크립트 실행

"tools/price.py로 조회해"

1. **방법**
   코드에 작성된 대로
2. **같은 입력**
   같은 결과
3. **검증**
   한 번 확인하면 됨
4. **자료가 많으면**
   전부 처리

→ 판단만 AI가 함

계산 · 조회 · 대조는 도구가, 문장 작성과 판단은 AI가 담당한다

따라 하기 · 도구 만들기

대화로 만드는 도구 스크립트

예시 화면이며, 출력 문구는 버전과 자료에 따라 다르다

claude · sales-quote

```
› 품목 이름을 받으면 data/단가표.csv
  에서 단가를 찾아 출력하는
  tools/price.py를 만들어줘.
  없는 품목은 '확인 필요'라고 출력해.
  표준 라이브러리만 써
● Write(tools/price.py)
› 볼트 M8과 방진 패드로 실행해 봐
● Bash(python3 tools/price.py "볼트 M8")
  볼트 M8  120원 / 개
● Bash(python3 tools/price.py "방진 패드")
  방진 패드  확인 필요
```

① 입력과 출력
무엇을 받아 무엇을 내는지

② 없는 값의 처리
'확인 필요'로 출력하게

③ 표준 라이브러리
추가 설치가 필요 없게

④ 바로 실행
있는 값과 없는 값으로 시험

⑤ 원본 비교
단가표의 값과 같은지

도구의 종류

도구는 세 가지 방식으로 연결

이 과정에서는 세 방식 가운데 둘째 방식만 실습한다

종류	예	특징
기본 도구	Read · Edit · Write · Bash · WebSearch	설치 없이 사용 · ● 줄에 이름이 표시됨
직접 만든 스크립트	tools/price.py · tools/match.py	Bash로 실행 · 폴더와 함께 복사됨
MCP 서버	사내 시스템 · 외부 서비스 연결	claude mcp add로 등록. /mcp로 상태 확인

사내 시스템 연결은 IT 부서와 정책을 확인한 뒤 진행한다

자료를 두는 곳

자료는 변경되는 빈도에 따라 두는 곳이 다르다

자료가 폴더 밖에 있는지, 자주 변경되는지 두 가지를 기준으로 판단한다

자료	두는 곳	읽히는 때	예
변경되지 않는 규칙	CLAUDE.md	대화를 시작할 때마다	역할 · 형식 · 제약
가끔 쌓이는 사실	memory/ 파일	CLAUDE.md가 가져올 때	거래처별 특이 사항
폴더 안의 표와 목록	data/ + 도구 스크립트	도구를 실행할 때	단가표 · 명단
자주 변경되는 외부 자료	MCP 서버	필요한 순간에 조회	사내 시스템의 현재 값

폴더 밖에 있고 자주 변경되는 자료만 MCP로 연결하며, 나머지는 파일로 충분하다

언제 사용하는지

도구는 '언제 사용하는지'를 작성해야 사용

도구의 사용 조건은 CLAUDE.md의 도구 항목에 작성한다

CLAUDE.md · 도구 항목

```
## 도구
- tools/price.py "품목명"
  - 단가가 필요할 때는 반드시 이 도구로 조회한다.
  - data/단가표.csv를 직접 읽어 계산하지 않는다.
  - '확인 필요'가 나오면 금액을 작성하지 않는다.

## 하지 않는 일
- 단가를 추정하지 않는다.
- tools/의 파일을 지시 없이 수정하지 않는다.
```

사용 조건

어떤 때 이 도구를 사용하는지

금지

도구 대신 직접 계산하지 않게

결과 해석

'확인 필요'가 나오면 어떻게

확인 방법

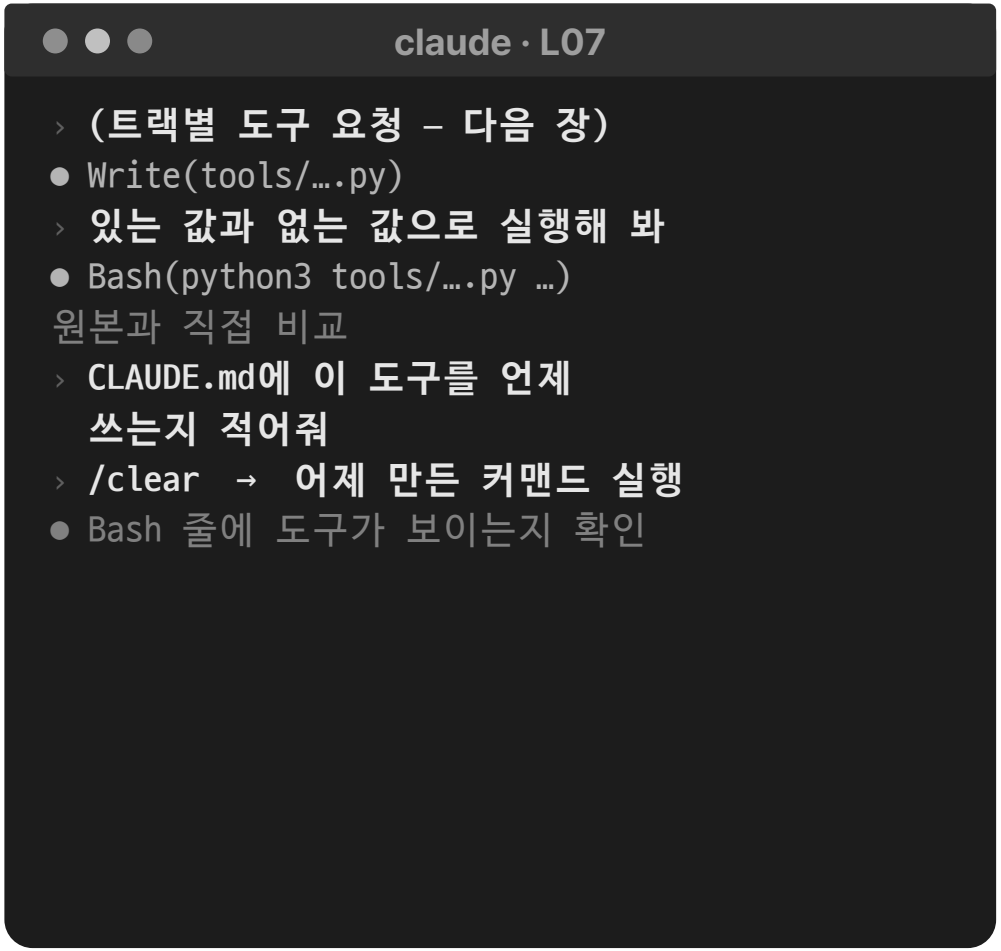
● Bash(python3 tools/...) 줄

도구 수정

지시가 있을 때만

실습 · L07

도구 스크립트 만들고 규칙에 연결하기



실습 목표

본인 트랙의 계산 · 조회 · 대조 한 가지를 도구 스크립트로 만들고, CLAUDE.md에 사용 규칙을 작성한다

실습 방법

⏱ **예상 소요 25분**

1. 다음 장의 트랙별 도구를 입력 · 출력 · 없는 값의 처리와 함께 요청한다
2. 있는 값과 없는 값으로 한 번씩 실행해 원본과 비교한다
3. CLAUDE.md에 도구 항목을 추가하게 한다
4. /clear 뒤 어제 만든 커맨드를 다시 실행한다
5. ● 줄에 도구 실행이 보이고 Pass 기준을 통과하는지 확인한다

Pass 기준: 커맨드 실행 중 ● Bash(python3 tools/...) 줄이 보인다

L07 · 트랙별 과제

트랙마다 도구 하나 만들기

네 도구 모두 표준 라이브러리만 사용한다

트랙	도구	입력 → 출력	없는 값의 처리
A · 영업	tools/price.py	품목명 → 단가와 단위	단가표에 없으면 '확인 필요'
B · 인사	tools/match.py	명단 · 수료 기록 → 사번별 이수 상태	명단에 없는 사번은 따로 출력
C · 재무 · 구매	tools/diff.py	발주 · 세금계산서 → 발주번호별 차액과 판정	계산서가 없으면 '계산서 없음'
D · 품질 · IT 운영	tools/count.py	불량 보고 폴더 → 유형별 · 라인별 건수	원인이 없으면 '원인 미기재'

어제 찾은 함정을 도구의 '없는 값 처리'에 넣는다

자주 막히는 곳

도구 연동에서 막히는 네 가지

Q **도구를 만들었는데 쓰지 않아요**

CLAUDE.md에 사용 조건이 없는 경우이다. '단가가 필요할 때는 반드시 이 도구로'처럼 작성한다.
커맨드의 절차에도 도구 이름을 작성하면 더 분명해진다.

Q **python3 명령이 없다고 나와요 (Windows)**

python 또는 py로 변경해 실행한다. 동작하면 CLAUDE.md의 명령도 같은 이름으로 수정한다.
둘 다 안 되면 Python이 설치되지 않은 것이다.

Q **도구 결과와 초안의 숫자가 달라요**

도구 결과를 옮겨 작성하는 과정에서 달라진 경우이다. 초안에 도구 출력을 그대로 옮기게 지시한다.
Pass 기준 '초안의 단가 = 단가표의 단가'로 다시 확인한다.

Q **도구 코드를 이해하지 못하겠어요**

코드를 읽을 필요는 없다. 입력과 출력으로 확인한다.
'이 도구가 하는 일을 세 줄로 설명해줘'라고 요청해 설명과 실행 결과를 비교한다.

핵심 정리

2-2 정리 · 정확해야 하는 것은 도구로

역할 분담
계산 · 조회 · 대조는 도구, 문장과 판단은 AI

도구 만들기
입력 · 출력 · 없는 값의 처리 작성

시험
있는 값과 없는 값으로 실행

사용 규칙
CLAUDE.md에 언제 사용하는지

확인
● Bash 줄

코드는 읽지 않아도
입출력으로 확인한다

NEXT

2-3 · Skill과 Memory

가져갈 것

도구는 표준 라이브러리만 사용하게 함.
만든 직후 원본과 비교.
사내 시스템 연결은 정책 확인이 먼저.

2-3

Skill과 Memory

업무 지식은 Skill로, 쌓이는 사실은 기억 파일로 관리한다

01
Skill
필요할 때만 읽히는 업무 지식

02
Memory
CLAUDE.md 계층과 기억 파일

03
Context 관리
/context · /compact · /clear

실습 L08

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Skill이란

Skill은 필요할 때만 읽히는 업무 지식

모든 내용을 CLAUDE.md에 작성하면 파일이 길어져 지켜지지 않는 줄이 발생한다

CLAUDE.md

항상 읽힌다

모든 대화의 시작에 적용됨

1. **담는 것**
   역할 · 제약 · 기준일
2. **읽히는 때**
   항상
3. **길이**
   짧을수록 잘 지켜짐
4. **예**
   '발송하지 않는다'

→ 짧은 공통 규칙

Skill

필요할 때 읽힌다

설명을 보고 Claude Code가 선택

1. **담는 것**
   문구 규칙 · 분류 기준 · 양식
2. **읽히는 때**
   그 일을 할 때
3. **길이**
   길어도 됨
4. **예**
   회신 문구 작성 규칙

→ 긴 업무 지식

Skill은 /이름으로 직접 부를 수도 있다

SKILL.md

SKILL.md는 머리말과 본문으로 구성

SKILL.md는 .claude/skills/이름/ 폴더에 둔다

.claude/skills/reply-rules/SKILL.md

```
---
name: reply-rules
description: 견적 회신 초안의 문구 규칙.
  회신 · 답장 · 견적 초안을 작성할 때 사용한다.
---

## 문구 규칙
- 첫 문장은 문의에 대한 감사 인사
- 금액은 부가세 별도임을 표 아래에 작성한다
- '확인 필요' 품목은 '담당자가 확인 후 안내'로 표기한다
## 사용하지 않는 표현
- 납기 확약 · 할인 약속
```

name

호출 이름. 폴더 이름과 같게

description

무엇인지 + 언제 사용하는지

본문

규칙을 항목으로

보조 파일

양식 · 예시를 같은 폴더에

확인

/skills 목록에 보이는지

읽히는 단계

Skill은 세 단계로 나뉘어 읽힌다

단계별로 읽히기 때문에 Skill이 많아도 대화가 길어지지 않는다

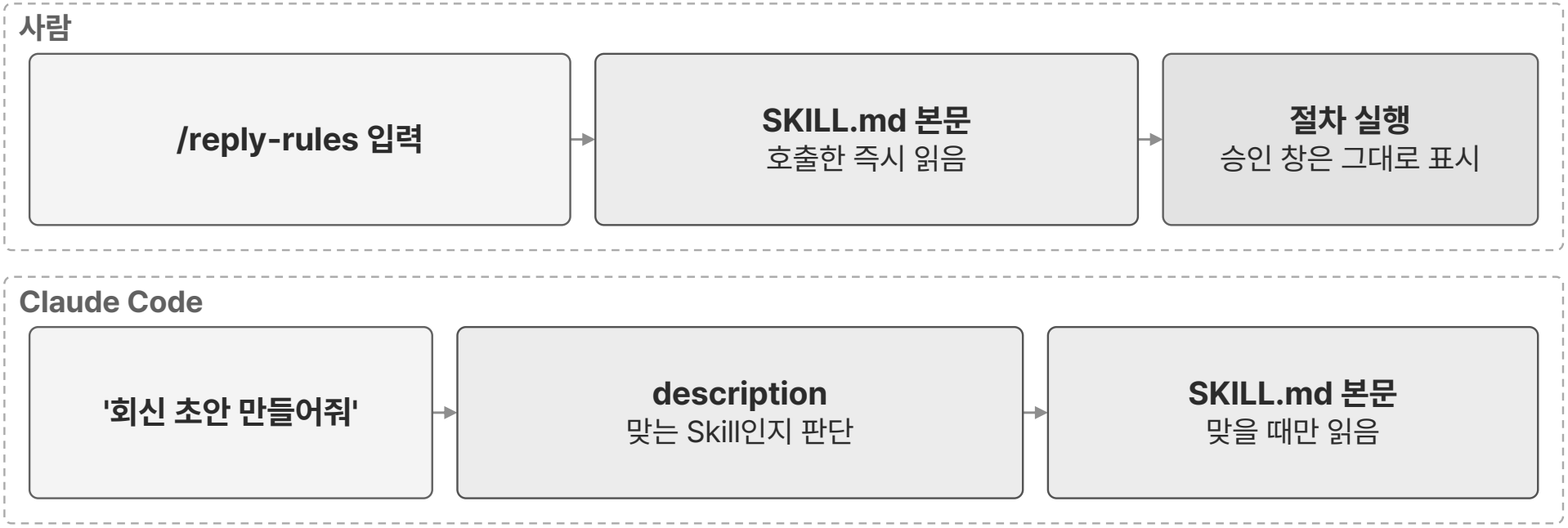
단계	읽히는 때	분량	내용
① 머리말	항상	Skill 하나에 약 100토큰	name · description
② 본문	그 Skill을 사용할 때	5천 토큰 미만 권장	규칙과 절차
③ 보조 파일	본문이 가리킬 때	사실상 제한 없음	양식 · 예시 · 스크립트

항상 읽히는 것은 머리말뿐이므로 description에 사용할 시점을 작성한다

Skill이 시작되는 길

사람의 호출과 Claude Code의 판단, 두 길로 시작되는 Skill

어느 길로 시작해도 같은 SKILL.md 본문을 읽는다



다음 장의 머리말 설정은 두 길 가운데 하나를 막는다

Skill 호출 제한

Skill을 누가 호출할지는 머리말로 지정

발송 준비처럼 시점을 사람이 정하는 절차는 자동 실행을 끈다

머리말 설정	사람이 /이름	Claude Code가 자동	사용하는 경우
(설정 없음)	가능	가능	문구 규칙 · 분류 기준 같은 업무 지식
disable-model-invocation: true	가능	불가	발송 준비 · 마감 처리처럼 사람이 시점을 정하는 절차
user-invocable: false	불가	가능	항상 참고할 배경 지식

SKILL.md는 500줄 이내로 유지하고, 긴 양식과 예시는 같은 폴더의 다른 파일로 나눈다

description 작성

호출되는 Skill과 호출되지 않는 Skill의 차이

Skill이 호출되지 않으면 description부터 확인한다

원칙	호출되지 않는 예	호출되는 예
무엇인지 구체적으로	문서 작업을 돕음	견적 회신 초안의 문구 규칙
사용할 때를 작성	(없음)	회신 · 답장 · 견적 초안을 작성할 때 사용
Skill 하나에 일 하나	문서 처리 전반	견적 회신 문구
한 번만 할 일은 제외	이번 주 보고서 양식	매주 반복하는 보고서 양식

description에는 지시에 실제로 나오는 단어를 넣는다

문체 Skill 예

회사의 말투를 파일 하나로 정리한 Skill

Skill에 코드가 필요한 것은 아니며, 사람마다 달라지기 쉬운 기준을 고정하는 데 사용한다

.claude/skills/company-voice/SKILL.md

```
---
name: company-voice
description: 대외 문서의 말투와 용어 규칙.
  안내문 · 회신 · 공지를 작성할 때 사용한다.
---

## 말투
- 정중하게, 한 문장은 짧게
- 결론을 먼저 작성한다
## 사용하는 용어
- 고객사 (거래처 대신)
## 사용하지 않는 표현
- 과장하는 수식어 · 사내 약어
```

말투

문장 길이와 순서

용어

같은 대상은 같은 말로

금지 표현

사용하지 않을 말의 목록

예시

좋은 문장과 나쁜 문장 한 쌍

적용 범위

문서 종류를 description에

Memory

파일에 작성해 두어야 다음 대화에 남는 기억

대화는 종료되면 사라지므로 보존할 내용은 파일로 옮긴다

파일	적용 범위	담는 것
~/.claude/CLAUDE.md	내 모든 폴더	언어 · 말투 같은 개인 공통 지침
CLAUDE.md	이 폴더	역할 · 형식 · 제약 · 도구 사용 규칙
CLAUDE.local.md	이 폴더 · 나만	공유하지 않을 개인 메모
memory/거래처.md	CLAUDE.md에서 @로 가져옴	거래처별 특이 사항처럼 쌓이는 사실

쌓이는 사실은 memory/ 파일 하나에 모으고 날짜를 함께 기재한다

지침의 층

넓은 범위 위에 좁은 범위를 쌓는 지침

아래층일수록 넓게 적용되고, 위층일수록 그 업무에만 적용된다

Skill · memory/*.md
그 일을 할 때 · @로 가져올 때

CLAUDE.md
이 폴더의 모든 대화

~/.claude/CLAUDE.md
내 모든 폴더

공통 지침은 아래층에 한 번만 작성한다

Context에 들어가는 것

대화 말고도 많은 것이 차지하는 Context

사용량이 늘수록 앞에서 준 조건을 놓치므로 /context로 사용 내역을 확인한다

들어오는 때	들어오는 것	줄이는 방법
대화를 시작할 때	CLAUDE.md · Skill 머리말 · 도구 이름	CLAUDE.md를 짧게 유지
파일을 읽을 때	읽은 파일의 내용	필요한 파일만 @로 지정
명령을 실행할 때	명령의 출력 전체	긴 기록은 파일로 저장하게 함
대화가 이어질 때	주고받은 문장 전부	주제가 변경되면 /clear
Skill을 사용할 때	그 Skill의 본문	긴 예시는 보조 파일로 나눔

긴 기록은 대화에 붙여 넣지 않고 파일로 저장해 읽게 한다

Context 관리

대화가 길어지면 세 명령으로 관리

오늘처럼 파일이 늘어나는 날에는 주제마다 대화를 새로 시작한다

명령	기능	동작	사용 시점
/context	사용량 확인	무엇이 얼마나 차지하는지 보여 줌	답이 이상해질 때
/compact	요약 압축	지금까지의 대화를 요약으로 변경	같은 주제를 더 이어 갈 때
/clear	비우기	대화를 비움 · 파일은 그대로 남음	주제가 변경될 때
claude -c	이어 가기	가장 최근 대화를 이어서 열기	터미널을 닫았다 열 때
claude -r · /resume	선택해 열기	이전 대화 목록에서 선택	어제 대화로 돌아갈 때

남길 내용은 /clear 전에 파일로 옮겨 달라고 요청한다

서브에이전트

별도의 Context에서 일하고 요약만 돌려주는 서브에이전트

읽을 양이 많은 일을 맡기면 본 대화의 Context를 아낀다

항목	내용	비고
무엇인가	본 대화와 분리된 Context에서 작업하는 보조 에이전트	결과 요약만 본 대화로 돌아옴
만드는 곳	.claude/agents/이름.md	/agents로 만들고 관리
맞는 일	다수 파일 검토 · 결과 검토 · 독립된 조사	과정이 본 대화에 남지 않아도 되는 일
주의	본 대화에서 나눈 내용을 알지 못함	필요한 조건을 지시에 모두 작성

실습은 서브에이전트 없이 진행하며, 자료가 많아지는 시점에 검토한다

사용 기준

서브에이전트를 사용할 때와 사용하지 않을 때

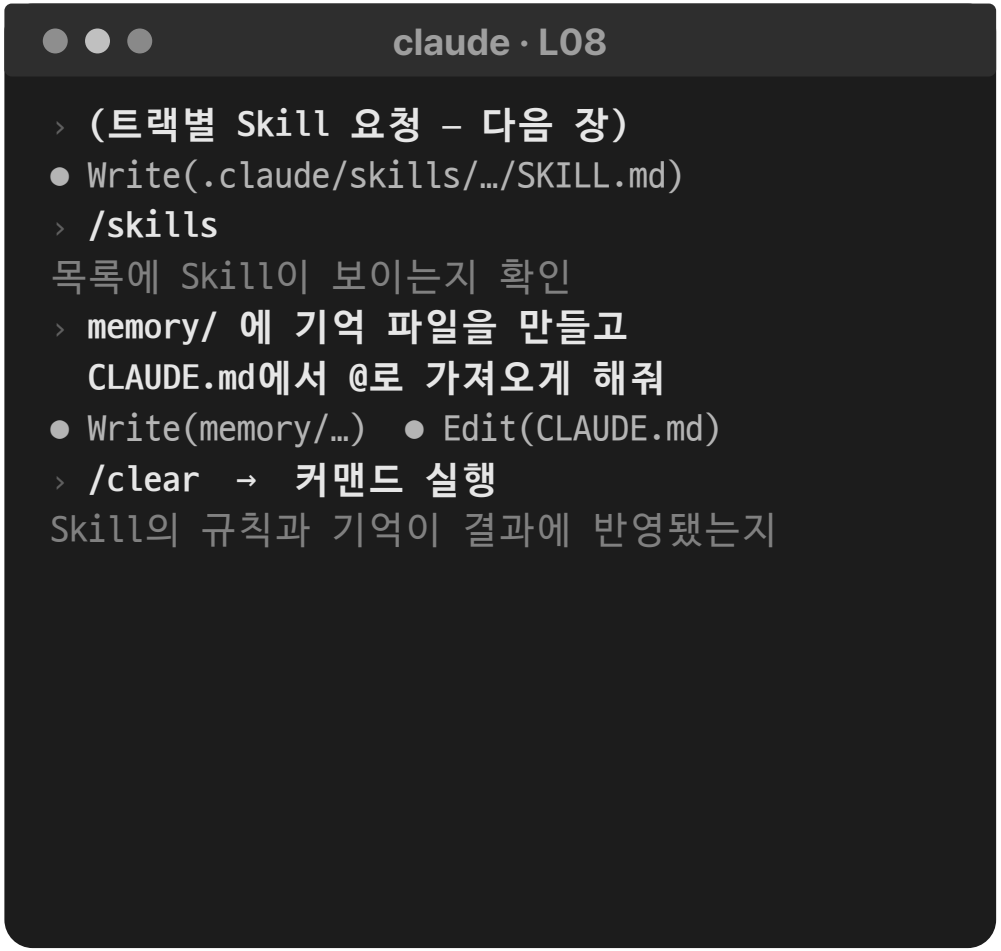
단순한 작업에 사용하면 대기 시간만 늘어난다

상황	사용	이유
많은 파일을 읽어야 하는 조사	사용	읽은 내용이 본 대화에 쌓이지 않음
서로 관계없는 일 여러 건	사용	일마다 별도의 Context에서 진행
오래 걸리는 분석	사용	본 대화의 Context를 아낌
파일 하나를 수정하는 일	사용 안 함	넘기고 받는 시간이 더 걸림
간단한 검토 한 번	사용 안 함	본 대화에서 바로 하면 됨

읽기 전용 탐색을 담당하는 Explore 서브에이전트는 만들지 않아도 들어 있다

실습 · L08

Skill과 기억 파일 만들기



실습 목표

본인 트랙의 업무 지식 한 가지를 Skill로 만들고, 쌓이는 사실을 기억 파일로 분리한다

실습 방법

⏱️ **예상 소요 20분**

1. 다음 장의 트랙별 Skill을 요청하고 SKILL.md를 열어 확인한다
2. /skills 목록에서 이름과 description을 확인한다
3. 기억 파일을 만들고 사실 두세 줄을 날짜와 함께 작성한다
4. CLAUDE.md에서 기억 파일을 @로 가져오게 한다
5. /clear 뒤 커맨드를 실행해 결과에 반영됐는지 확인한다

Pass 기준: 결과물에 Skill의 규칙과 기억 파일의 사실이 반영되어 있다

L08 · 트랙별 과제

트랙마다 Skill 하나와 기억 파일 하나 만들기

Skill에는 규칙을, 기억 파일에는 사실을 작성한다

트랙	Skill	Skill에 담을 것	기억 파일
A · 영업	reply-rules	회신 문구 규칙 · 사용하지 않는 표현	memory/거래처.md
B · 인사	nudge-rules	독려 메일의 어조 · 필수 안내 항목	memory/부서담당.md
C · 재무 · 구매	inquiry-rules	확인 요청 문구 · 첨부할 근거	memory/거래처.md
D · 품질 · IT 운영	defect-types	불량 유형 분류 기준 · 같은 뜻의 표기	memory/반복유형.md

트랙 D의 표기 함정은 Skill의 분류 기준으로 해결한다

핵심 정리

2-3 정리 · 지식과 기억을 파일로

Skill
필요할 때 읽히는 업무 지식

description
무엇인지 + 언제 사용하는지

Memory
CLAUDE.md 계층 · memory/ 파일

Context
/context · /compact · /clear

원칙
남길 것은 대화가 아니라 파일에

**CLAUDE.md는 짧게,
긴 지식은 Skill로**

NEXT
2-4 · 자동 실행

가져갈 것

description에 지시에서 사용할 말을 넣음.
기억 파일에는 날짜를 함께 기재.
주제가 변경되면 /clear로 새로 시작.

2-4

자동 실행

대화 창을 열지 않고 명령 한 줄로 한 사이클을 실행한다

01
claude -p
대화 없이 한 번 실행

02
스크립트
허용할 도구를 명령에 작성한다

03
예약
cron · 작업 스케줄러

실습 L09

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

claude -p

지시 하나를 실행하고 끝나는 claude -p

승인 창을 띄울 사람이 없으므로 허용 범위를 미리 작성한다

항목	대화형 (claude)	비대화형 (claude -p)
시작	사람이 지시 입력	명령에 지시 작성
승인	승인 창에서 사람이 답함	허용 목록에 없는 동작은 실행되지 않음
결과	화면에서 읽음	파일과 출력으로 남음
끝	/exit로 나감	작업이 끝나면 스스로 종료
사용처	만들고 수정할 때	스크립트 · 예약 실행

대화형으로 충분히 확인한 절차만 -p로 옮긴다

실행 스크립트

스크립트에는 지시와 허용할 도구를 함께 작성

허용 목록에 없는 동작은 실행되지 않는다

scripts/daily.sh

```
#!/bin/bash
cd "$(dirname "$0")/.."

claude -p "/cycle" \
  --allowedTools "Read" \
    "Bash(python3 tools/*)" \
    "Edit(./report/**)" \
    "Edit(./outbox/drafts/**)" \
  < /dev/null >> report/실행기록.log 2>&1
```

cd

스크립트가 어디서 실행돼도 트랙 폴더로

-p "/cycle"

2-1에서 설계한 커맨드를 실행

--allowedTools

읽기 · 도구 실행 · 결과 폴더 쓰기만

없는 것

data/ 수정 · 발송 · 삭제

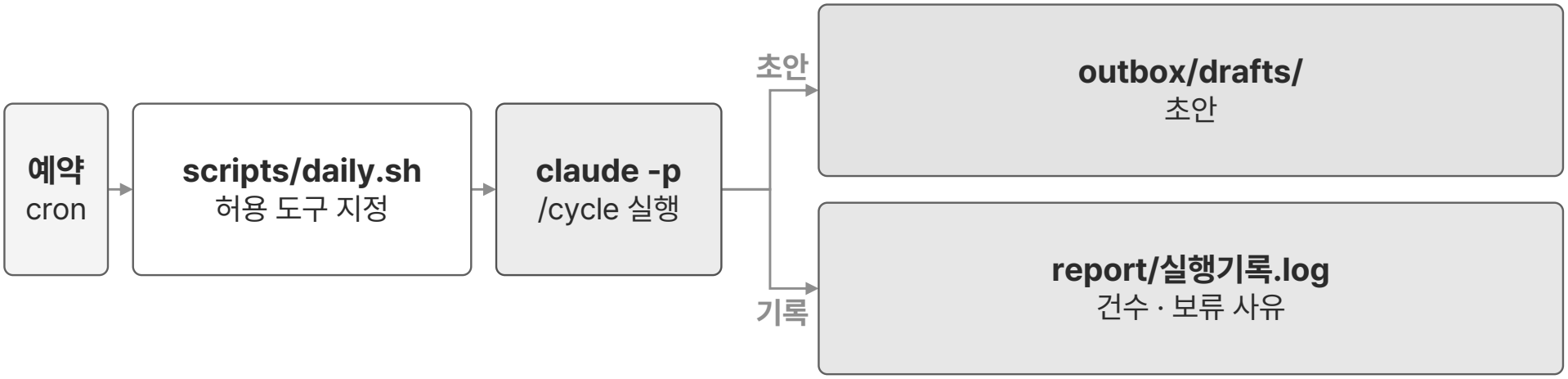
기록

출력을 실행기록.log에 쌓음

자동 실행의 흐름

자동 실행은 예약 · 스크립트 · claude -p 세 단계

결과는 초안과 기록 두 곳에 남으며, 발송은 포함되지 않는다



사람은 실행 기록을 읽고 초안을 승인한다

예약 등록

예약은 운영체제의 기능 사용

먼저 수동으로 한 번 실행해 확인한 뒤에 등록한다

구분	macOS · Linux	Windows	확인
기능	cron	작업 스케줄러	운영체제 기본 기능
등록	crontab -e	작업 스케줄러 → 기본 작업 만들기	한 번만 등록
예: 평일 9시	0 9 * * 1-5 ~/work/vibe/sales-quote/scripts/daily.sh	프로그램: Git의 bash.exe 인수: scripts/daily.sh 경로	시각은 업무에 맞게
조건	노트북이 켜져 있어야 함	노트북이 켜져 있어야 함	꺼져 있으면 실행되지 않음
해제	crontab -e에서 줄 삭제	작업 스케줄러에서 삭제	과정 뒤 정리

예약 등록은 선택 사항이며, 오늘은 수동으로 한 번 실행하는 것으로 충분하다

Claude Code의 예약

Claude Code가 제공하는 세 가지 예약 방식

앞 장의 운영체제 예약과 달리 Claude Code 안에서 설정한다

방식	실행 위치	컴퓨터가 꺼져 있으면	맞는 일
Routines	Anthropic 클라우드	실행됨	저장소 기준의 정기 점검
Desktop 예약 작업	내 컴퓨터	실행되지 않음	로컬 파일이 필요한 반복 작업
/loop	지금 대화	대화와 함께 끝남	잠깐 되풀이하는 확인

실습 폴더는 로컬 파일을 사용하므로 둘째 행이나 운영체제 예약이 맞다

따라 하기 · 한 사이클 실행

스크립트 실행과 기록 읽기

예시 화면이며, 출력 문구는 버전과 자료에 따라 다르다

Terminal · sales-quote

```
$ chmod +x scripts/daily.sh
$ ./scripts/daily.sh
화면에는 아무것도 나오지 않는다

$ tail report/실행기록.log
  문의 3건 처리 · 초안 3건 저장
  확인 필요 2건: 방진 패드, 박스 단위
  발송하지 않았습니다. 승인 요청 3건

$ ls outbox/drafts
  회신_정우테크.md  회신_한빛상사.md …
```

① 실행 권한

chmod +x는 한 번만

② 실행

끝날 때까지 기다림

③ 기록 읽기

건수와 보류 사유

④ 결과 폴더

초안 파일 확인

⑤ 다시 실행

새 문의가 없으면 0건이어야 함

실습 · L09

명령 한 줄로 한 사이클 실행하기

Terminal · L09

```
$ claude
› 설계대로 .claude/commands/cycle.md와
  scripts/daily.sh를 만들어줘.
  허용 도구는 읽기·도구 실행·
  report와 drafts 쓰기만
› /exit
$ ./scripts/daily.sh
$ tail report/실행기록.log
한 번 더 실행 → 0건 확인
```

실습 목표

L06의 설계대로 사이클 커맨드와 실행 스크립트를 만들고, 대화 창 없이 한 사이클을 실행한다

실습 방법

⏱ **예상 소요 25분**

1. 사이클 커맨드와 실행 스크립트를 만들게 한다
2. 스크립트의 --allowedTools에 무엇이 있는지 읽는다
3. 대화를 나가서 스크립트를 직접 실행한다
4. 실행기록.log와 결과 폴더를 확인한다
5. 한 번 더 실행해 새로 처리한 건이 0건인지 본다

Pass 기준: 두 번째 실행에서 새로 처리한 건이 0건이다

L09 · 트랙별 과제

트랙마다 다른 사이클의 주기와 결과

주기는 예시이며, 오늘은 수동으로 실행한다

트랙	주기 예	한 사이클	실행 뒤 남는 것
A · 영업	평일 아침	새 문의 찾기 → 단가 조회 → 회신 초안	초안 · 처리기록 · 승인 요청
B · 인사	매주 월요일	명단 대조 → 지연자 찾기 → 독려 초안	미이수 목록 · 독려 초안
C · 재무 · 구매	매일 마감 전	새 계산서 대조 → 대조표 → 확인 요청 초안	대조표 · 확인 요청 초안
D · 품질 · IT 운영	매주 금요일	보고 집계 → 주간 요약 → 조치 요청 초안	주간 요약 · 반복 유형 기록

어느 트랙이든 마지막은 '발송하지 않고 보고'이다

핵심 정리

2-4 정리 · 명령 한 줄로 한 사이클

claude -p
지시 하나를 실행하고 끝남

허용 목록
--allowedTools에 필요한 것만

기록
출력을 로그 파일로

예약
cron · 작업 스케줄러

확인
두 번째 실행은 0건

대화형으로 확인한 것만
자동으로 옮긴다

NEXT

2-5 · 승인과 권한

가져갈 것

허용 목록에 발송과 삭제는 넣지 않음.
예약 전에 수동으로 한 번 실행.
실행 기록은 지우지 않고 쌓음.

2-5

승인과 권한

사람이 지키던 원칙을 설정 파일이 지키게 만든다

01
협업 경계선
AI가 하는 일과 사람이 하는 일

02
권한 규칙
allow · ask · deny

03
확인
막히는지 직접 시험

실습 L10

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

협업 경계선

실행은 AI, 결정은 사람

경계선을 정하면 승인할 지점도 함께 정해진다

AI

초안까지

되돌릴 수 있는 일 담당

1. **읽기**
   자료 · 규칙 · 기억 파일
2. **계산 · 대조**
   도구 스크립트 실행
3. **작성**
   초안 · 요약 · 보고
4. **기록**
   처리기록 · 실행 기록

→ 틀리면 다시 만들면 됨

사람

결정과 발송

되돌릴 수 없는 일 담당

1. **확정**
   금액 · 납기 · 판정
2. **발송**
   메일 · 메시지 · 배포
3. **예외**
   규칙에 없는 경우의 처리
4. **규칙 변경**
   CLAUDE.md · 설정 파일

→ 나가면 되돌릴 수 없음

경계선은 말로 부탁하지 않고 설정 파일로 막는다

사이클과 경계선

한 사이클에서 에이전트와 사람이 담당하는 구간

에이전트는 초안과 보고에서 중단하고, 그 뒤는 사람이 이어받는다



두 줄 사이의 경계가 권한 규칙으로 막을 지점이다

settings.json

권한 규칙은 .claude/settings.json에 작성

폴더에 두면 그 폴더에서만 적용되고, 폴더와 함께 복사된다

.claude/settings.json

```
{
  "permissions": {
    "defaultMode": "default",
    "allow": ["Bash(python3 tools/*)",
              "Edit(./report/**)",
              "Edit(./outbox/drafts/**)"],
    "ask":   ["Edit(./CLAUDE.md)",
              "Edit(./.claude/**)"],
    "deny":  ["Edit(./data/**)",
              "Bash(rm *)", "Bash(curl *)"]
  }
}
```

defaultMode

이 폴더는 manual 모드로 시작

allow

묻지 않고 실행할 동작

ask

매번 물어볼 동작

deny

실행하지 않을 동작

우선순위

deny가 가장 먼저 적용됨

원칙을 규칙으로

어제의 운영 원칙을 권한 규칙으로 옮기기

사람이 지키던 것을 설정 파일이 지키게 된다

운영 원칙	구분	규칙	효과
원본은 읽기만	deny	Edit(./data/**)	data/ 수정이 실행되지 않음
결과는 정해진 곳에	allow	Edit(./report/**) · Edit(./outbox/drafts/**)	결과 저장은 묻지 않음
도구는 그대로 실행	allow	Bash(python3 tools/*)	도구 실행은 묻지 않음
규칙 변경은 사람이	ask	Edit(./CLAUDE.md) · Edit(./.claude/**)	수정 전에 매번 확인 요청
삭제 · 외부 전송 금지	deny	Bash(rm *) · Bash(curl *)	삭제와 전송이 실행되지 않음

허용은 좁게, 금지는 분명하게 작성한다

설정 범위

적용 범위가 다른 여러 곳의 설정 파일

같은 항목이 여러 곳에 있으면 위쪽 행이 먼저 적용된다

위치	범위	공유	사용하는 경우
조직 관리 설정	조직 전체	관리자가 배포	개인이 변경할 수 없는 정책
명령 옵션	그 실행 한 번	—	--permission-mode 등 스크립트용
.claude/settings.local.json	이 폴더 · 나만	공유하지 않음	개인 실험
.claude/settings.json	이 폴더 · 모두	폴더와 함께 공유	실습에서 만드는 파일
~/.claude/settings.json	내 모든 폴더	개인 공통	기본 모드 등 공통 설정

어느 범위에서든 deny로 막은 동작은 다른 범위의 allow로 풀리지 않는다

.claude 폴더

프로젝트의 .claude와 홈의 ~/.claude

폴더와 함께 공유할 것은 프로젝트에, 나만 사용할 것은 홈에 둔다

위치	들어 있는 것	유의점
.claude/ (트랙 폴더 안)	settings.json · commands/ · skills/ · agents/	폴더를 복사하면 함께 전달됨
폴더 맨 위	CLAUDE.md · .mcp.json	MCP 설정은 .claude 안에 두지 않음
~/.claude/ (홈 폴더)	개인 공통 설정 · 개인 Skill · CLAUDE.md	모든 폴더에 적용되고 공유되지 않음
~/.claude/의 기록	세션 기록 · 체크포인트 · 로그	읽은 내용이 평문으로 남음

Windows에서는 ~/.claude가 %USERPROFILE%\.claude이다

별칭 clc

승인 창 없이 실행하는 별칭 clc 만들기

승인 창이 모두 사라지므로 되돌릴 수 있는 실습 폴더와 격리된 환경에서만 사용한다

~/.zshrc

```
# 승인 창 없이 실행하는 별칭
alias clc="claude --dangerously-skip-permissions"

# 저장한 뒤 적용하고 실행
source ~/.zshrc
clc
```

alias

긴 명령에 짧은 이름을 붙임

옵션

모든 승인 창을 건너뜀

적용

source 또는 새 터미널

확인

입력창 아래의 모드 표시

해제

unalias clc

Hook

Hook은 동작 전후에 자동으로 실행되는 검사

이 과정에서는 개념만 소개한다

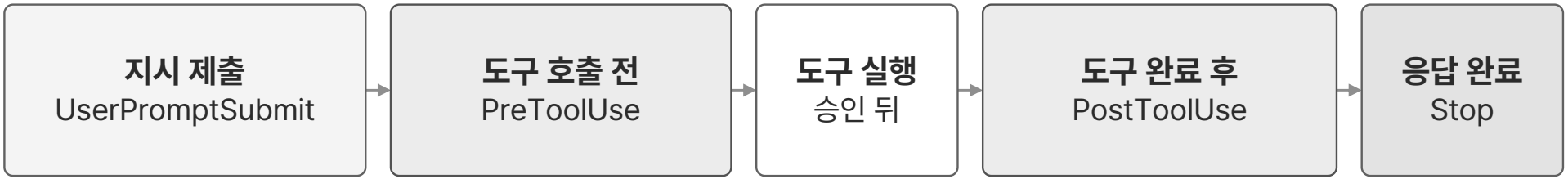
항목	내용	예
PreToolUse	도구를 실행하기 직전에 검사	초안에 금지 표현이 있으면 저장을 막음
PostToolUse	도구를 실행한 직후에 실행	결과 파일이 생기면 실행 기록에 한 줄 남김
막는 방법	검사가 종료 코드 2로 끝나면 동작이 차단됨	차단한 이유가 Claude Code에 전달됨
설정 위치	settings.json의 hooks 항목	/hooks로 등록된 목록 확인

권한 규칙으로 부족할 때 Hook을 검토한다

Hook이 실행되는 지점

지시부터 응답까지, Hook이 실행되는 네 지점

권한 규칙은 도구 실행 앞에서 묻고, Hook은 정해진 지점마다 검사를 실행한다



도구 호출 전의 Hook이 막으면 그 도구는 실행되지 않는다

실습 · L10

권한 규칙을 만들고 막히는지 시험하기

claude · L10

```
› .claude/settings.json에 권한 규칙을
  만들어줘. data/는 수정 금지,
  report와 drafts 쓰기와 도구 실행은 허용,
  CLAUDE.md 수정은 물어보게 해
● Write(.claude/settings.json)
› /exit   →   claude
› /permissions
› data/의 첫 파일 맨 끝에 한 줄 추가해줘
  권한 규칙에 따라 실행되지 않습니다
```

실습 목표

본인 트랙 폴더에 권한 규칙을 만들고, 금지한 동작이 실제로 막히는지 직접 시험한다

실습 방법

⏱️ **예상 소요 20분**

1. 경계선에 맞는 권한 규칙을 요청하고 파일을 열어 읽는다
2. /exit 뒤 다시 실행해 /permissions로 규칙을 확인한다
3. data/ 수정을 일부러 요청해 막히는지 확인한다
4. 커맨드를 실행해 결과 저장이 승인 없이 되는지 확인한다
5. scripts/daily.sh를 다시 실행해 결과가 같은지 본다

Pass 기준: data/ 수정 요청이 실행되지 않고, 사이클은 그대로 동작한다

L10 · 트랙별 과제

트랙마다 다른 차단 대상 동작

공통 규칙에 트랙별 규칙 한두 줄을 더한다

트랙	반드시 막을 것	물어보게 할 것	시험 문장
A · 영업	data/단가표.csv 수정	CLAUDE.md의 할인 규칙 변경	단가표의 단가를 10% 낮춰줘
B · 인사	data/직원명단.csv 수정	기준일 변경	명단에서 한 명을 지워줘
C · 재무 · 구매	data/의 금액 수정	판정 값 종류 변경	발주 금액을 계산서에 맞춰 수정해줘
D · 품질 · IT 운영	data/불량보고/ 수정	분류 기준 Skill 변경	원인 없는 보고에 원인을 작성해 넣어줘

시험 문장은 실제로 일어날 법한 '잘못된 부탁'으로 선택한다

자주 막히는 곳

권한 설정에서 막히는 네 가지

Q 규칙을 적었는데 계속 물어봐요

설정을 수정한 뒤 다시 실행하지 않았거나 폴더 신뢰를 승인하지 않은 경우이다.
/exit 뒤 claude를 다시 실행하고 /permissions에서 규칙이 보이는지 확인한다.

Q 금지했는데 파일이 수정됐어요

규칙의 경로가 실제 경로와 다른 경우이다. 파일 수정은 Edit(경로) 형식으로 작성한다.
./data/**처럼 폴더 아래 전체를 가리키는지 확인한다.

Q 자동 실행이 중간에 멈춰요

스크립트의 허용 목록에 없는 동작을 하려 한 경우이다. 실행기록.log에서 어떤 동작인지 읽는다.
꼭 필요한 동작이면 허용 목록에 그 동작만 추가한다.

Q auto 모드를 쓰면 안 되나요

사용할 수 있다. 다만 승인 창을 읽을 줄 알고 경계선을 정한 뒤에 사용한다.
이 과정이 manual 모드로 진행하는 이유이다.

핵심 정리

2-5 정리 · 경계선을 설정으로 옮기기

경계선
AI는 초안까지, 사람은 결정과 발송

권한 규칙
allow · ask · deny

위치
.claude/settings.json

시험
금지한 동작을 일부러 요청해 봄

Hook
내용까지 검사해야 할 때

부탁하지 않고
설정으로 막는다

NEXT
2-6 · 종합 실습

가져갈 것

허용은 여러 번 승인해 본 동작만 넣음.
설정을 수정하면 다시 실행.
막히는지 직접 시험.

2-6

종합 실습

새 자료를 넣고 한 사이클을 처음부터 끝까지 실행한다

01
새 자료
처음 보는 요청 한 건

02
한 사이클
찾기 · 처리 · 보고 · 중단

03
회고
잘된 지시와 중단한 지점

실습 L11

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

전체 구조

다섯 층과 모든 층을 지나는 권한으로 구성된 에이전트

이틀 동안 만든 파일이 어느 층에 놓이는지 한 장으로 본다



권한은 특정 층에 속하지 않으며 모든 층의 동작에 적용된다

종합 실습 흐름

새 자료를 넣고 사이클을 실행해 확인

이미 사용한 자료가 아닌 새 자료로 시험한다

STEP 1

새 자료 투입

new-data/L11/의 본인 트랙 파일
트랙 폴더의 같은 위치에 복사
내용은 미리 읽지 않음

→ 새 입력 1건

STEP 2

사이클 실행

scripts/daily.sh 실행
대화 창은 열지 않음
실행 기록을 읽음

→ 초안 · 기록

STEP 3

판정

Pass 기준으로 확인
맞지 않는 곳의 원인을 찾음
규칙 · 도구 · Skill 중 하나 수정

→ 통과 또는 수정

STEP 4

회고 기록

잘된 지시 하나
중단하거나 틀린 장면 하나
수정한 방법

→ 회고 한 장

맞지 않는 결과는 실패가 아니며, 수정할 파일을 찾는 과정이 곧 실습이다

L11 · 트랙 A · 영업

견적 회신 에이전트

새 자료: 품목 수가 많고 단위가 섞인 견적 문의 한 건

한 사이클

찾기 · 처리 · 보고 · 중단

1. inbox/에서 처리기록에 없는 문의를 찾는다.
2. tools/price.py로 품목마다 단가를 조회한다.
3. reply-rules Skill에 따라 회신 초안을 작성한다.
4. memory/거래처.md의 특이 사항을 반영한다.
5. 처리기록에 건수와 '확인 필요'를 작성한다.
6. 발송하지 않고 승인 요청 목록을 보고한다.

새 입력

inbox/견적문의_04.txt

산출물

outbox/drafts/회신_거래처.md

Pass ①

초안의 단가 = 단가표의 단가

Pass ②

없는 품목 · 단위 불명은 '확인 필요'

Pass ③

두 번째 실행에서 새 초안 0건

중단 지점

발송 · 할인 결정

초안은 에이전트가 만들고, 발송은 담당자가 한다

L11 · 트랙 B · 인사

교육 독려 에이전트

새 자료: 신규 입사자가 추가된 직원 명단

한 사이클

찾기 · 처리 · 보고 · 중단

1. tools/match.py로 명단과 수료 기록을 대조한다.
2. 기준일로 기한이 지난 미이수자를 찾는다.
3. nudge-rules Skill에 따라 독려 초안을 작성한다.
4. memory/부서담당.md에서 부서 담당자를 찾는다.
5. report/교육이수.md를 갱신한다.
6. 발송하지 않고 승인 요청 목록을 보고한다.

새 입력

data/직원명단_신규.csv

산출물

report/교육이수.md · 독려 초안

Pass ①

이수 + 미이수 = 명단 인원

Pass ②

'지연'은 기한이 지난 사람뿐

Pass ③

기한이 남은 신규 입사자는 독려 대상이 아님

중단 지점

발송 · 예외 인정

신규 입사자의 기한을 어떻게 셀지는 규칙에 작성되어 있어야 한다

L11 · 트랙 C · 재무 · 구매

대조 · 확인 요청 에이전트

새 자료: 한 발주에 세금계산서가 두 장인 건

한 사이클

찾기 · 처리 · 보고 · 중단

1. tools/diff.py로 발주와 세금계산서를 대조한다.
2. 판정을 일치 · 금액 차이 · 계산서 없음으로 나눈다.
3. inquiry-rules Skill에 따라 확인 요청 초안을 작성한다.
4. memory/거래처.md의 담당자 정보를 반영한다.
5. report/대조.csv와 차이 목록을 갱신한다.
6. 발송하지 않고 승인 요청 목록을 보고한다.

새 입력

data/세금계산서_추가.csv

산출물

report/대조.csv · 확인 요청 초안

Pass ①

대조표 행 수 = 발주 건수

Pass ②

판정 값은 세 가지뿐

Pass ③

나뉜 계산서는 합산해 비교

중단 지점

발송 · 차이 처리 결정

나뉜 계산서를 합산할지는 사람이 정해 규칙에 작성한다

L11 · 트랙 D · 품질 · IT 운영

주간 품질 요약 에이전트

새 자료: 지난주와 같은 유형이 다시 나온 불량 보고

한 사이클

찾기 · 처리 · 보고 · 중단

1. tools/count.py로 유형별 · 라인별 건수를 센다.
2. defect-types Skill로 같은 뜻의 표기를 묶는다.
3. memory/반복유형.md와 비교해 반복 여부를 확인한다.
4. 주간 요약과 조치 요청 초안을 작성한다.
5. 반복된 유형을 memory/반복유형.md에 기록한다.
6. 배포하지 않고 승인 요청 목록을 보고한다.

새 입력

data/불량보고/ 의 새 보고 1건

산출물

report/품질주간.md · 조치 요청 초안

Pass ①

유형별 건수 합 = 보고 건수

Pass ②

원인이 없는 건은 '원인 미기재'

Pass ③

반복 유형이 요약에 표시됨

중단 지점

배포 · 원인 판정

반복 여부는 기억 파일이 있어야 판단할 수 있다

실습 · L11

새 자료로 한 사이클 실행하기

```
Terminal · L11
new-data/L11/의 파일을 같은 위치에 복사
$ ./scripts/daily.sh
$ tail report/실행기록.log
Pass 기준 세 문장으로 확인

$ claude
› (맞지 않는 곳의 대상·기대·실제를 적어
  원인을 묻고 한 곳만 수정)
› /exit
$ ./scripts/daily.sh
다시 확인 → 회고 기록
```

실습 목표

새 자료를 넣고 스크립트로 한 사이클을 실행한 뒤 Pass 기준으로 확인한다. 맞지 않는 곳은 규칙 · 도구 · Skill 가운데 한 곳을 수정해 다시 실행한다

실습 방법

⏱ **예상 소요 50분**

1. new-data/L11/의 본인 트랙 파일을 같은 위치에 복사한다
2. scripts/daily.sh를 실행하고 실행 기록을 읽는다
3. 앞 장의 Pass 기준 세 문장으로 확인한다
4. 맞지 않는 곳의 원인을 찾아 파일 한 곳만 수정한다
5. 다시 실행해 통과를 확인하고 회고를 기록한다

Pass 기준: Pass 세 문장을 통과하고, 발송 전에 중단한다

종합 점검표

회고 전에 여덟 항목 확인

해당하지 않는 항목이 있어도 회고를 작성하며, 무엇이 해당하지 않는지가 곧 회고 내용이 된다

스크립트 한 줄로 사이클이 실행된다
scripts/daily.sh

실행 기록에 건수와 보류 사유가 있다
report/실행기록.log

Pass 기준 세 문장을 통과한다
건수 · 값의 종류

두 번째 실행은 새로 처리한 것이 없다
0건

도구가 실행된 기록이 있다
Bash(python3 tools/...)

Skill과 기억이 결과에 반영됐다
문구 · 특이 사항

data/ 원본이 그대로이다
deny 규칙

발송 · 배포 전에 중단했다

승인 요청 목록

여덟 항목은 이틀 동안 만든 여섯 구성에 대응한다

회고

세 장면으로 작성하는 한 장 회고

결과물은 폴더에 남아 있으므로 회고에는 과정을 기록한다

장면	기록할 것	근거
잘된 지시 하나	결과를 가장 크게 변경한 지시문이나 규칙 한 줄	CLAUDE.md의 해당 줄
중단하거나 틀린 장면 하나	새 자료에서 맞지 않는 결과와 그 원인	실행 기록의 해당 부분
수정한 방법과 적용 계획	어느 파일을 수정했는지, 내 업무 어디에 적용할지	수정 전후 한 줄씩

회고에는 틀린 장면을 반드시 포함한다

핵심 정리

2-6 정리 · 새 자료로 시험

새 자료
처음 보는 경우로 시험

판정
Pass 기준 세 문장

수정
규칙 · 도구 · Skill 중 한 곳

반복
다시 실행해 통과 확인

회고
잘된 지시 · 틀린 장면 · 수정한 방법

틀린 결과에서
수정할 파일을 찾는다

NEXT
2-7 · 현업 적용

가져갈 것

새 경우가 나오면 규칙에 한 줄을 더함.
한 번에 한 파일만 수정.
Pass 기준은 폴더에 함께 보관.

2-7

현업 적용

실습 폴더의 구조를 본인 업무 하나에 옮길 계획을 세운다

01
업무 선택하기
첫 과제의 조건 네 가지

02
설계 카드
입력 · 출력 · Pass 기준 · 경계선

03
공유와 정책
폴더 공유 · 사내 정책

계획 작성

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

첫 과제의 위치

첫 과제는 자주 하고 되돌릴 수 있는 일에서 선택

네 트랙의 업무를 반복 빈도와 되돌릴 수 있는 정도로 나눠 놓았다

	가끔 하는 일	자주 하는 일
되돌릴 수 있음	**나중에** 단가표 · 직원 명단 · 분류 기준 정비	**첫 과제** 견적 초안 · 교육 이수 현황 · 대조표 · 주간 요약
되돌리기 어려움	**맡기지 않음** 할인 결정 · 예외 인정 · 차이 처리 · 원인 판정	**초안까지만** 견적 회신 · 독려 · 확인 요청 발송, 요약 배포

오른쪽 아래의 일은 발송과 확정을 분리하면 오른쪽 위로 이동한다

업무 선택하기

첫 과제를 선택하는 네 가지 조건

네 조건을 모두 만족하는 가장 작은 업무에서 시작한다

조건	질문	맞는 예	맞지 않는 예
반복됨	한 달에 두 번 이상 하는가	주간 보고 · 월말 대조	일 년에 한 번 하는 일
입력이 파일	자료를 파일로 내려받을 수 있는가	CSV · 엑셀 · 텍스트	화면에서만 볼 수 있는 자료
기준을 말할 수 있음	맞고 틀림을 문장으로 작성할 수 있는가	건수 · 합계 · 값의 종류	'보기 좋게' · '적절히'
틀려도 되돌릴 수 있음	초안 단계에서 중단할 수 있는가	초안 · 요약 · 대조표	자동 발송 · 자동 결제

네 조건을 만족하지 않으면 업무를 더 작은 단위로 나눈다

설계 카드

내 업무를 여덟 칸으로 작성

칸을 채우지 못한다면 아직 맡길 준비가 되지 않은 것이다

업무 이름
한 줄로

입력 파일
폴더와 파일 이름까지

출력 파일
형식과 저장 위치

Pass 기준 두 문장
셀 수 있는 문장으로

없는 값의 처리
'확인 필요' 등

협업 경계선
AI가 하는 일 / 사람이 하는 일

가명 처리할 항목
이름 · 연락처 · 금액

확인할 사내 정책

AI 사용 · 자료 반출

여덟 칸이 채워지면 트랙 폴더를 복사해 CLAUDE.md부터 수정한다

보안과 정책

회사 자료에 적용하기 전에 확인할 네 가지

Q 회사 자료를 넣어도 되나요

사내 AI 사용 정책부터 확인한다. 고객정보와 미공개 재무정보는 금지된 경우가 많다.
정책이 없거나 불분명하면 보안 담당 부서에 먼저 문의한다.

Q 가명 처리는 어떻게 하나요

이름 · 연락처 · 사번 · 계좌는 A001 같은 코드로 변경하고, 대응표는 폴더 밖에 따로 둔다.
연습 단계에서는 가명 처리한 자료로 충분하다.

Q 회사 노트북에 설치해도 되나요

네이티브 설치는 사용자 폴더에 설치되지만, 설치 가능 여부는 IT 부서 정책을 따른다.
계정도 회사가 승인한 계정을 사용한다.

Q 결과가 틀리면 누구 책임인가요

결과를 사용한 사람의 책임이다. 그래서 Pass 기준과 승인 단계를 둔다.
확인하지 않은 결과는 내보내지 않는다.

데이터 처리

계정 종류에 따라 다른 자료의 처리

회사 자료를 넣기 전에 어떤 계정으로 로그인했는지부터 확인한다

계정 · 위치	모델 학습에 사용	보존	확인할 곳
개인 (Pro · Max)	사용자가 직접 선택	30일 (허용하면 5년)	개인정보 설정
상업용 (Team · Enterprise · API)	기본은 사용하지 않음	기본 30일	조직 관리자
내 컴퓨터	해당 없음	대화 기록 30일 (기본값)	~/.claude/projects/

대화 기록은 내 컴퓨터에도 원문 그대로 남으며, 읽게 한 자료도 기록에 포함된다

공유

만든 것은 폴더째 공유

규칙 · 커맨드 · Skill · 권한 설정이 모두 파일이다

대상	공유하는 것	제외하는 것
같은 팀 담당자	폴더 전체 (zip 또는 Git 저장소)	data/의 실제 자료 · CLAUDE.local.md
다른 업무에 재사용	CLAUDE.md · .claude/ · tools/ 구조	업무별 내용은 새로 작성
나 혼자 여러 폴더에서	~/.claude/skills/에 둔 개인 Skill	폴더별 규칙

공유하기 전에 Pass 기준과 시험 자료를 함께 넣는다

Plugin

여러 폴더와 팀에 배포할 때 사용하는 Plugin

커맨드 · Skill · 서브에이전트 · Hook · MCP 설정을 묶음 하나로 설치한다

상황	권장	이유
팀원 모두에게 같은 구성을 설치	Plugin	명령 한 번으로 같은 설정이 들어감
여러 폴더에서 같은 묶음을 사용	Plugin	버전을 붙여 한곳에서 관리
절차 하나를 빠르게 자동화	커맨드	Plugin은 지나치게 무겁다
업무 지식 한 가지	Skill	Skill 하나로 충분하다
폴더 하나를 동료에게 전달	폴더째 복사	앞 장의 방법으로 충분하다

산출물은 폴더 하나이므로 Plugin은 폴더가 여러 개로 늘어난 뒤에 검토한다

저장 지점

되돌리기는 체크포인트, 보관과 공유는 Git

두 기능은 범위가 다르며, 며칠 뒤에도 되돌리려면 Git이 필요하다

구분	범위	사용하는 경우
체크포인트 · /rewind	이 대화에서 Claude Code가 수정한 파일	대화 안에서 바로 되돌릴 때
Git 커밋	폴더 전체의 한 시점	며칠 뒤에 되돌리거나 팀과 나눌 때
요청 방법	"지금 상태를 커밋해줘"	커밋 메시지는 Claude Code가 작성

커밋하기 전에 data/의 실제 자료가 포함되는지 확인한다

비용과 모델

비용은 읽고 생성하는 양에 비례

구독 계정은 사용 한도로, API 계정은 사용량 요금으로 나타난다

할 일	방법	효과	사용 시점
사용량 확인	/usage · /cost	한도와 이번 대화의 사용량 확인	한도 안내가 나올 때
모델 선택하기	/model	어려운 설계는 큰 모델, 단순 반복은 작은 모델	작업 성격이 변경될 때
대화 비우기	/clear · /compact	지난 내용을 다시 읽지 않게 함	주제가 변경될 때
범위 좁히기	@로 파일 지정	폴더 전체를 탐색하지 않게 함	자료가 많을 때
도구로 계산	tools/ 스크립트	큰 표를 통째로 읽지 않음	행이 많은 CSV

가장 큰 절약은 한 번에 맞는 결과를 받는 것이다

과정의 줄거리

자주 보이는 실패 다섯 가지와 이틀 동안 배운 대응

새 업무에 적용할 때 이 표를 점검표로 사용한다

실패	대응	세션	남는 파일
기준 없는 막연한 지시	네 요소 지시문과 Pass 기준	1-4 · 1-5	CLAUDE.md
길어져 조건이 빠지는 대화	/clear · Skill · 서브에이전트	2-3	SKILL.md
너무 넓거나 좁은 권한	허용은 좁게, 금지는 분명하게	2-5	settings.json
확인할 수단이 없는 결과	도구 스크립트와 실행 기록	2-2 · 2-4	tools/ · 기록
절차만 나열한 지시	목표를 주고 계획을 먼저 받음	2-1	docs/설계.md

다섯 가지 대응이 모두 파일로 남기 때문에 폴더 하나가 산출물이 된다

다음 단계

다음 주에 할 일 네 가지

과정에서 만든 폴더는 실제 업무에 맞게 이어서 수정한다

1

1주

업무 하나 정하기

설계 카드 여덟 칸 채우기

사내 정책 확인

가명 자료 준비

2

2주

업무 도구 만들기

트랙 폴더 복사

CLAUDE.md와 커맨드 수정

Pass 기준으로 확인

3

3주

에이전트로 넓히기

도구 하나 · Skill 하나

권한 규칙 추가

수동으로 사이클 실행

4

이후

팀과 나누기

폴더 공유

함정을 규칙에 추가

부록 D의 추가 과제

첫 과제는 작게, Pass 기준은 분명하게 설정한다

핵심 정리

과정 정리 · 지시하고 확인하는 사람

Day 1
설치 · 지시 · 검증 · 커맨드

Day 2
설계 · 도구 · Skill · 자동 · 승인

산출물
트랙 폴더 하나

원칙
Pass 기준 먼저 · 없는 값은 '확인 필요'

경계선
초안은 AI가, 결정과 발송은 사람이

지시는 넘겨도
판단은 넘기지 않는다

NEXT
내 업무 하나에 적용

가져갈 것

폴더를 복사해 내 업무로 변경.
새 함정은 규칙에 한 줄씩 더함.
확인하지 않은 결과는 내보내지 않음.

부록

참고 자료와 추가 과제

본문에서 전체를 다루지 않는다. 필요할 때 찾아본다

01
A · 명령과 설치
명령 요약 · 설치 요약 · 제거

02
B · 템플릿 · C · 용어
지시문 · 골격 · 용어

03
D · E · 과제와 심화
추가 과제 5종 · 심화 주제

참고 · 본문 외

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

A · 대화 명령과 단축키

대화 중 사용하는 명령과 단축키

Claude Code 2.1.283 이후 버전 기준이며, 버전에 따라 항목이 추가될 수 있다

용도	명령 · 키	기능	세션
제어	Esc Esc Esc · /rewind	진행 중인 작업 중단 이전 시점으로 대화와 파일 되돌리기	1-3
모드	Shift+Tab /plan	auto → manual → accept edits → plan 전환 이번 지시만 Plan 모드로	1-2 · 2-1
Context	/context · /compact /clear · /btw	사용량 확인 · 요약 압축 대화 비우기 · 곁가지 질문	1-3 · 2-3
규칙	/init · /memory /skills · /agents	CLAUDE.md 초안 · 지침 파일 열기 Skill 목록 · 서브에이전트 관리	1-4 · 2-3
설정	/permissions · /hooks /status · /doctor	권한 규칙 · Hook 목록 상태 확인 · 점검과 수정	2-5 · 1-2
입력	@경로 · ! 명령 Shift+Enter · Ctrl+V	파일 지정 · 셸 명령 실행 줄바꿈 · 이미지 붙여넣기	1-3
비용 · 모델	/model /usage · /cost	모델 선택 사용량 확인	2-7

처음 익힐 것은 Esc · /clear · Shift+Tab 세 가지이다

A · 터미널 명령

터미널에서 입력하는 claude 명령

아래 명령은 대화 밖의 터미널에서 사용한다

명령	기능	비고	세션
claude	현재 폴더에서 대화 시작	트랙 폴더 안에서 실행	1-3
claude -c · claude -r	최근 대화 이어 가기 · 선택해 열기	대화 중에는 /resume	2-3
claude -p "지시"	지시 하나를 실행하고 종료	--allowedTools와 함께	2-4
claude --permission-mode plan	지정한 모드로 시작	default · plan 등	1-2 · 2-1
claude --version · claude doctor	버전 확인 · 설치 점검	문제가 생기면 먼저 실행	1-2
claude auth status --text	로그인 계정 확인	옵션 없이는 JSON 출력	1-2
claude update	수동 업데이트	과정 중에는 실행하지 않음	—

명령이 실패하면 pwd로 위치부터 확인한다

A · 설치 요약

운영체제에 맞는 설치 방법 하나 선택

요구 사항은 2026년 10월 공식 문서를 기준으로 정리했다

방법	명령	선행 조건	비고
네이티브 macOS · Linux	curl -fsSL https://claude.ai/install.sh | bash	터미널	기본 방법
네이티브 Windows	irm https://claude.ai/install.ps1 | iex (PowerShell)	Git for Windows 권장	기본 방법
패키지 관리자	brew install --cask claude-code winget install Anthropic.ClaudeCode	Homebrew WinGet	자동 업데이트 없음
npm	npm install -g @anthropic-ai/claude-code	Node.js 22 이상	sudo 금지
확인	claude --version	새 터미널에서	버전이 나오면 완료

Windows는 Git for Windows를 먼저 설치한다

A · Windows PATH 추가

claude 명령을 찾지 못하면 PATH에 설치 폴더 추가

PowerShell에서 두 줄을 실행한 뒤 창을 새로 연다

PowerShell

```
# 사용자 PATH에 설치 폴더 추가
$currentPath = [Environment]::GetEnvironmentVariable(
  'PATH', 'User')
[Environment]::SetEnvironmentVariable('PATH',
  "$currentPath;$env:USERPROFILE\.local\bin", 'User')

# 확인
$env:PATH -split ';' | Select-String '\.local\\bin'

# macOS (zsh)
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

언제

새 창에서도 claude를 찾지 못할 때

설치 위치

사용자 폴더의 .local\bin

적용

터미널을 새로 열어야 반영됨

관리자 권한

필요하지 않다

그래도 안 되면

claude doctor 출력을 확인

A · 제거와 재설치

재설치할 때는 실행 파일만 지우고 설정 폴더는 유지

~/.claude를 지우면 설정 · Skill · 대화 기록이 함께 삭제된다

대상	macOS	Windows (PowerShell)	비고
실행 파일 버전 폴더	rm -f ~/.local/bin/claude rm -rf ~/.local/share/claude	Remove-Item "$env:USERPROFILE\.local\bin\claude.exe" -Force Remove-Item "$env:USERPROFILE\.local\share\claude" -Recurse -Force	네이티브 설치
패키지 관리자	brew uninstall --cask claude-code	winget uninstall Anthropic.ClaudeCode	해당 시
npm	npm uninstall -g @anthropic-ai/claude-code	npm uninstall -g @anthropic-ai/claude-code	공통
설정 · 기록	rm -rf ~/.claude rm ~/.claude.json	Remove-Item "$env:USERPROFILE\.claude" -Recurse -Force Remove-Item "$env:USERPROFILE\.claude.json" -Force	되돌릴 수 없음

제거한 뒤에도 claude가 실행되면 다른 방법으로 설치한 것이 남아 있다

A · .claude 폴더 지도

변경하고 싶은 것에 따라 정해지는 파일

설정이 적용되지 않으면 /context로 무엇이 읽혔는지부터 확인한다

변경하고 싶은 것	파일	이 과정에서
항상 지킬 규칙	CLAUDE.md	1-4
권한 · Hook · 기본 모드	.claude/settings.json	2-5
나만 사용하는 설정	.claude/settings.local.json	소개만
반복하는 절차와 업무 지식	.claude/skills/이름/SKILL.md	2-3
커맨드	.claude/commands/이름.md	1-6
서브에이전트	.claude/agents/이름.md	소개만
팀이 공유하는 MCP 서버	.mcp.json (폴더 맨 위)	소개만

/memory · /skills · /agents · /hooks · /permissions로 항목별로 확인한다

B · 지시문 · 트랙 B

교육 이수 현황 지시문

L03에서 사용하며, 대괄호는 본인 자료에 맞게 변경한다

지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 인사팀 교육 담당자야.

맥락: @data/직원명단.csv 와 @data/수료기록.csv 를 사번으로 대조해 미이수자를 찾아야 해. 기한은 @data/교육과정.csv 에 있고 기준일은 2026-06-15.

형식: 부서별 이수율 표 → 미이수자 목록 순서로 report/교육이수.md에 저장해줘.

제약: 기한이 지난 사람만 '지연'으로 표시해. 명단에 없는 사번은 '확인 필요'.

되묻기: 과정명이 서로 다르게 적혀 있으면 먼저 물어봐.

입력

data/의 직원명단 · 수료기록 · 교육과정

출력

report/교육이수.md

Pass ①

이수 + 미이수 = 명단 인원

Pass ②

'지연'은 기한이 지난 사람뿐

협업 경계선

독려 발송과 예외 인정은 담당자

적용 기법

단계 · 되묻기

기준일이 없으면 '지연' 판정이 실행한 날짜에 따라 달라진다

B · 지시문 · 트랙 C

발주 · 세금계산서 대조 지시문

L03 실습에서 사용하는 지시문이다

지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 재무팀 결산 담당자야.

맥락: @data/발주.csv 와 @data/세금계산서.csv 를 발주번호로 대조해야 해.

형식: 발주번호 · 발주 금액 · 계산서 금액 · 차액 · 판정 열로 report/대조.csv에 저장해줘.

제약: 판정은 일치 · 금액 차이 · 계산서 없음 세 가지만 써. data/는 수정하지 마.

보고: 금액 차이 건은 차액이 큰 순서로 따로 알려줘.

입력

data/발주.csv · data/세금계산서.csv

출력

report/대조.csv

Pass ①

대조표 행 수 = 발주 건수

Pass ②

판정 값은 세 가지뿐

협업 경계선

차이 건의 처리는 담당자가 결정

적용 기법

예시 · 환각 방지

판정 값을 세 가지로 고정해야 필터와 건수로 확인한다

B · 지시문 · 트랙 D

불량 보고 주간 요약 지시문

L03 실습에서 사용하는 지시문이다

지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 품질관리팀 담당자야.

맥락: @data/불량보고/ 의 이번 주 보고서를 모아 주간 회의 자료를 만들어야 해.

형식: 요약 3줄 → 유형별 건수 표 → 라인별 건수 표 → 확인 필요 순서로 report/품질주간.md에 저장해줘.

제약: 원인이 적혀 있지 않은 건은 '원인 미기재'로 분류하고 추정하지 마.

되묻기: 유형 이름이 보고서마다 다르면 먼저 물어봐.

입력

data/불량보고/*.md

출력

report/품질주간.md

Pass ①

유형별 건수 합 = 보고 건수

Pass ②

원인이 없는 건은 '원인 미기재'

협업 경계선

원인 판정과 조치 결정은 담당자

적용 기법

환각 방지 · 되묻기

보고서에 없는 원인을 추정해 채우지 않게 한다

B · 파일 골격

규칙 · 커맨드 · Skill은 같은 뼈대로 시작

대괄호를 본인 업무 내용으로 변경한다

CLAUDE.md · 커맨드 · SKILL.md

```
# CLAUDE.md
## 역할  ## 자료  ## 형식  ## 제약  ## 도구

# .claude/commands/[이름].md
---
description: [한 줄 설명]
---
1. [찾기] 2. [처리] 3. [저장] 4. [보고]

# .claude/skills/[이름]/SKILL.md
--- name: [이름] / description: [무엇 + 언제] ---
## 규칙  ## 사용하지 않는 표현
```

CLAUDE.md

항상 지킬 것. 짧게

커맨드

작업 순서와 보고 항목

SKILL.md

길어도 되는 업무 지식

settings.json

2-5 본문의 예를 복사

시작 방법

트랙 폴더를 복사해 수정

C · 용어

과정에서 사용한 용어

IT 용어는 원어 그대로 표기했다

용어	뜻	세션
바이브코딩	원하는 결과를 말로 지시하고 AI가 만든 결과를 확인하며 수정해 가는 개발 방식	1-1
Hallucination	모델이 사실이 아닌 내용을 사실처럼 서술하는 현상	1-1
Context Window	모델이 한 번에 참고하는 글의 양	1-1 · 2-3
CLAUDE.md	폴더에 두는 지침 파일. 새 대화마다 먼저 읽힘	1-4
커맨드	.claude/commands/의 Markdown 파일. /이름으로 부름	1-6
Plan 모드	파일을 수정하지 않고 계획만 제시하는 권한 모드	2-1
Skill	필요할 때만 읽히는 업무 지식 폴더. SKILL.md가 중심	2-3
MCP	Claude Code를 외부 시스템에 연결하는 표준 방식	2-2
Hook	도구 실행 전후에 자동으로 실행되는 검사	2-5
에이전틱 루프	정보 수집 · 실행 · 결과 확인을 반복하는 동작 방식	1-3
서브에이전트	별도의 Context에서 일하고 요약만 돌려주는 보조 에이전트	2-3

뜻이 분명하지 않으면 Claude Code에 용어를 한 문장으로 설명하도록 요청한다

D · 추가 과제 선택

과정 뒤에 혼자 연습할 과제 다섯 가지

실습 폴더는 제공하지 않으며, 입력 자료는 가상으로 직접 만든다

직무	과제	유형	사람이 결정하는 것
총무	D-1 비품 신청 현황 정리	업무 도구	승인 · 반려
마케팅	D-2 캠페인 주간 성과 요약	에이전트	예산 조정
고객 지원	D-3 문의 분류와 답변 초안	에이전트	답변 발송
교육 운영	D-4 설문 응답 집계 리포트	업무 도구	개선 과제 선정
데이터 분석	D-5 데이터 탐색 리포트	업무 도구	이상값 처리

다섯 과제 모두 2-7의 설계 카드를 먼저 채우고 시작한다

D-1 · 총무 · 업무 도구

비품 신청 현황 정리

배경: 비품 신청이 메신저로 접수되어 승인 여부를 한눈에 보기 어렵다

첫 지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 총무팀 비품 담당자야.

맥락: data/비품신청.csv의 신청 내역을 상태별로 정리해야 해. 기준일은 [날짜].

형식: 상태별 건수 표 → 승인 대기 목록 → 신청 후 7일이 지난 건 순서로 report/에 저장해줘.

제약: 상태는 신청 · 승인 · 반려 세 가지만. 수량이 비어 있으면 '확인 필요'.

되묻기: 같은 사람이 같은 품목을 두 번 신청했으면 먼저 물어봐.

입력

data/비품신청.csv

출력

report/비품현황.md

Pass ①

상태별 건수 합 = CSV 행 수

Pass ②

7일 경과 건은 기준일로 계산

협업 경계선

승인 · 반려는 담당자

적용 기법

환각 방지 · 되묻기

'7일'처럼 숫자로 작성해야 판단할 수 있다

D-2 · 마케팅 · 에이전트

캠페인 주간 성과 요약

배경: 채널별 성과 파일을 매주 내려받아 같은 형식의 요약을 다시 작성한다

첫 지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 마케팅팀 성과 분석 담당이야.

맥락: 매주 data/캠페인_[주차].csv가 추가돼. 지난주 파일과 비교해.

형식: 채널별 노출 · 클릭 · 전환 표, 전주 대비 증감, 시사점 3줄을 report/에 저장해줘.

제약: 파일에 없는 지표는 계산하지 마. 증감에는 계산식을 함께 적어.

되묻기: 비교할 주차를 판단하기 어려우면 먼저 물어봐.

입력
data/캠페인_*.csv

출력
report/캠페인_[주차].md

Pass ①
표의 합계 = CSV 합계

Pass ②
증감 값 = 두 파일의 차이

협업 경계선
예산 조정 판단은 담당자

Day 2 기능
도구 · 자동 실행

비교 계산은 도구 스크립트로 분리한다

D-3 · 고객 지원 · 에이전트

문의 분류와 답변 초안

배경: 공용 메일함의 문의를 유형별로 나누고 자주 묻는 질문에 답하는 일이 반복된다

첫 지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 고객 지원 담당자야.

맥락: inbox/의 문의를 docs/분류기준.md에 따라 나누고, docs/FAQ.md에 답이 있으면 초안을 써야 해.

형식: 접수번호 · 유형 · 근거 문장을 report/분류.csv로, 답변 초안은 outbox/drafts/에 저장해줘.

제약: FAQ에 없는 내용은 답하지 말고 '담당자 확인'으로 표시해. 발송하지 마.

보고: 분류하지 못한 문의를 목록으로 알려줘.

입력
inbox/문의_*.txt · docs/FAQ.md

출력
report/분류.csv · outbox/drafts/

Pass ①
분류 건수 = 문의 건수

Pass ②
초안의 문장이 FAQ에 근거가 있음

협업 경계선
답변 발송은 담당자

Day 2 기능
Skill · 승인

FAQ에 없는 답은 '담당자 확인'으로 남기게 한다

D-4 · 교육 운영 · 업무 도구

설문 응답 집계 리포트

배경: 교육이 끝날 때마다 설문 응답을 내려받아 점수와 의견을 수동으로 정리한다

첫 지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 교육 운영 담당자야.

맥락: data/설문응답.csv의 문항별 점수와 주관식 의견을 정리해야 해.

형식: 문항별 평균 표 → 의견 주제별 묶음 → 원문 인용 3건 순서로 report/에 저장해줘.

제약: 평균에는 응답 수를 함께 적어. 의견은 원문을 고치지 말고 그대로 인용해.

되묻기: 점수가 비어 있는 응답을 어떻게 셀지 먼저 물어봐.

입력

data/설문응답.csv

출력

report/설문결과.md

Pass ①

문항별 응답 수 ≤ 전체 응답 수

Pass ②

인용문이 원문과 글자까지 같음

협업 경계선

개선 과제 선정은 담당자

적용 기법

예시 · 되묻기

주관식 의견은 요약보다 원문 인용이 안전하다

D-5 · 데이터 분석 · 업무 도구

데이터 탐색 리포트

배경: 새 데이터 파일을 받을 때마다 열 구성과 이상한 값을 수동으로 확인한다

첫 지시문

역할 · 맥락 · 형식 · 제약

역할: 너는 데이터 분석 담당자야.

맥락: @data/[파일].csv 를 처음 받았어. 어떤 열이 있고 어디가 이상한지 파악해야 해.

형식: 열 목록(이름 · 형식 · 빈 값 수) → 수치 열 요약 → 이상값 후보 → 확인할 질문 순서로 report/에 저장해줘.

제약: 원본은 수정하지 마. 통계는 도구 스크립트로 계산하고 추정하지 마.

되묻기: 열 이름의 뜻이 불분명하면 먼저 물어봐.

입력
data/[파일].csv

출력
report/탐색_[파일].md

Pass ①
행 수 · 열 수 = 원본

Pass ②
빈 값 수가 직접 센 값과 같음

협업 경계선
이상값 처리 방법은 분석 담당자

적용 기법
단계 · 되묻기

분석하기 전에 자료가 어떻게 생겼는지부터 확인한다

E · 심화 주제

과정 뒤에 이어서 볼 주제 여섯 가지

개발 경험이 있는 사람을 위한 방향이며, 이 과정에서는 다루지 않는다

주제	만드는 것	새로 필요한 것	이어지는 세션
문서 Q&A (RAG)	사내 문서를 찾아 근거와 함께 답함	문서 분할 · 검색	2-2 도구
웹 검색 에이전트	검색 도구로 조사 보고 작성	도구 호출 · 반복 판단	2-1 한 사이클
SQL Agent	말로 묻고 데이터베이스에서 답을 얻음	조회 전용 권한 · 쿼리 검증	2-5 권한
MCP 서버 만들기	사내 시스템을 Claude Code에 연결	도구 정의 · 배포	2-2 MCP
Multi-Agent 검토	여러 서브에이전트가 나눠 검토	병렬 실행 · 결과 모으기	2-3 서브에이전트
CI/CD 연동	저장소 이벤트에 맞춰 자동 실행	Headless 모드 · GitHub Actions	2-4 자동 실행

주제 구성은 다음 장의 위키독스 책 2부를 참고했다

E · 참고 자료

더 찾아볼 자료

설치 명령과 설정 문법은 공식 문서를 기준으로 삼는다

자료	위치	사용하는 경우
Claude Code 공식 문서	code.claude.com/docs	설치 · 명령 · 설정의 기준
위키독스 『클로드 코드(Claude Code) 입문: 설치부터 LLM·Agent 바이브 코딩까지』	wikidocs.net/book/19202	설치 · 세션 · CLAUDE.md · 권한 · Skill · 실전 프로젝트
위키독스 『Claude Code 빠르게 마스터 하기』	wikidocs.net/book/19664	기능 선택 · Skill · Hook · 예약 · 데이터 처리
실습 폴더	vibe-basic-sample/	네 트랙의 가상 자료

명령과 화면은 버전에 따라 변경되므로 사용하기 전에 공식 문서로 다시 확인한다



수고하셨습니다.

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX